Opaline Attachment Defense System for Proactive Detection and Sanitization of Malicious Email Files

Abstract

E-mail remains a primary mode of digital communication, and attachments play a crucial role in transmitting documents, reports, and media files. However, these attachments often serve as an entry point for cyberattacks, where malicious PDFs, Office documents, or scripts exploit hidden vulnerabilities the moment a user opens them. Traditional defenses depend heavily on signaturebased malware scanners and user awareness training, which are increasingly inadequate against zero-day exploits, polymorphic malware, and sophisticated spear-phishing attachments. As attackers continue to embed harmful payloads within seemingly legitimate documents, there is a pressing need for a more intelligent and proactive defense mechanism. To address this challenge, this project introduces Opaline Attachment, an advanced attachment-screening system powered by the DistilBERT deep learning model. DistilBERT analyzes both structural features and embedded textual semantics from incoming email attachments to detect suspicious patterns that may indicate hidden threats. Instead of relying solely on detection, the system incorporates an additional protective layer: potentially harmful attachments are isolated in a secured sandbox and transformed into safe, static PNG renderings. This rendering process removes executable components and embedded scripts, allowing users to preview the content without interacting with the original file. The system automatically substitutes the user's attachment view with its sanitized PNG version and provides cautionary alerts before granting access to the original file. By combining intelligent threat classification, secure content rendering, and controlled access, Opaline Attachment ensures robust protection against both known and unknown attachment-based attacks. This approach significantly reduces user exposure to harmful files and strengthens overall email security without disrupting normal workflow.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1. OVERVIEW

An email attachment is a file sent along with an email message, typically containing documents, images, videos, or software programs. These attachments can be in various formats, such as PDFs, Word documents, JPEG images, or ZIP files. While email attachments are widely used for legitimate purposes, they can also serve as a vector for cyberattacks, making them a significant security concern. Attackers often exploit email attachments to deliver malware, steal sensitive information, or gain unauthorized access to systems.

One common type of attack involving email attachments is malware distribution, where malicious software is embedded in the attachment. Upon opening, this software can install harmful programs, such as ransomware, trojans, or spyware, onto the recipient's system. Phishing attacks are also frequently carried out via email attachments, where attackers impersonate legitimate entities and trick users into opening malicious files that steal login credentials or financial information. Zero-day exploits can be hidden in attachments, targeting unpatched vulnerabilities in software applications, enabling attackers to compromise systems without detection. Social engineering tactics are often used to manipulate recipients into opening seemingly harmless attachments, such as invoices or urgent updates, which, in reality, contain malicious payloads. Additionally, macro viruses embedded in document files can execute harmful scripts when opened, often leading to widespread infections within an organization. Spyware and keyloggers can also be delivered via email attachments, enabling attackers to monitor activities or steal sensitive data, while Denial of Service (DoS) attacks can result from attachments designed to overload or crash systems.

1.2. PROBLEMS DEFINITION

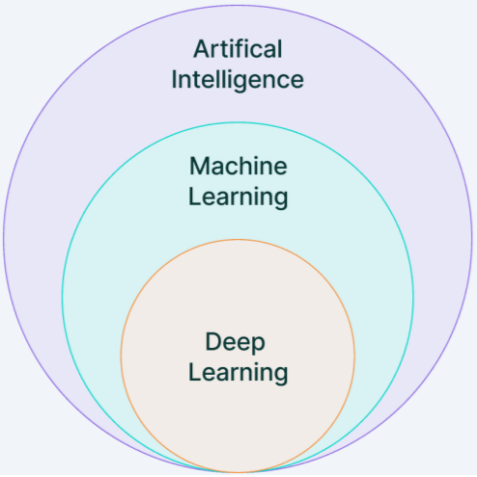
Email attachments are a critical component of digital communication but have become a major cybersecurity risk due to their potential to carry malicious payloads. Attackers exploit vulnerabilities in commonly used document formats such as PDFs, Word files, and Excel spreadsheets to execute malware, ransomware, or phishing attacks. These threats can be triggered merely by opening an infected attachment, putting users and organizations at significant risk. Traditional email security solutions rely on signature-based malware detection and user awareness training to mitigate these threats. However, these approaches have notable limitations:

- **Ineffectiveness Against Zero-Day Attacks** – Signature-based detection systems struggle to identify newly emerging or previously unknown malware variants.
- **Evasion Techniques** – Attackers use sophisticated obfuscation and polymorphic techniques to bypass traditional malware scanners.
- **Over-Reliance on User Awareness** – While training programs can reduce risk, they do not eliminate the possibility of human error, as users may still unknowingly interact with malicious attachments.
- **Potential False Positives and False Negatives** – Existing solutions may misclassify benign files as threats, leading to unnecessary disruptions, or fail to detect advanced threats, leaving systems vulnerable.

To address these challenges, this project proposes CrystallineAttachment, a novel approach that integrates machine learning-based detection (DistilBERT) with secure attachment rendering in a sandboxed environment. By analyzing the content and metadata of attachments using deep learning techniques and converting potentially harmful files into static PNG images, the system provides a proactive defense mechanism that minimizes the risk of exposure to malicious attachments. This project aims to enhance email security by ensuring that users can preview attachments safely without the risk of executing hidden malware, thereby reducing the impact of phishing and ransomware attacks while maintaining seamless usability.

1.3. MACHINE LEARNING

Machine learning is a subset of Artificial Intelligence (AI) that enables computers to learn from data and make predictions without being explicitly programmed. If you're new to this field, this tutorial will provide a comprehensive understanding of machine learning, its types, algorithms, tools, and practical applications.



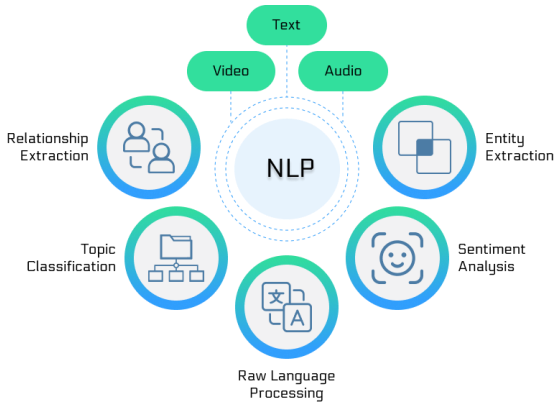
Machine learning teaches computers to recognize patterns and make decisions automatically using data and algorithms.

- Supervised Learning: Trains models on labeled data to predict or classify new, unseen data.
- Unsupervised Learning: Finds patterns or groups in unlabeled data, like clustering or dimensionality reduction.
- Reinforcement Learning: Learns through trial and error to maximize rewards, ideal for decision-making tasks.

1.3.1. Natural Language Processing

Natural language processing (NLP) is a field of computer science and a subfield of artificial intelligence that aims to make computers understand human language. NLP uses computational linguistics, which is the study of how language works, and various models based on statistics, machine learning, and deep learning. These technologies allow computers to analyze and process text or voice data, and to grasp their full meaning, including the speaker’s or writer’s intentions and emotions. NLP powers many applications that use language, such as text translation, voice recognition, text summarization, and chatbots. You may have used some of these applications yourself, such as voice-operated GPS systems, digital assistants, speech-to-text software, and

customer service bots. NLP also helps businesses improve their efficiency, productivity, and performance by simplifying complex tasks that involve language.



NLP Techniques

NLP encompasses a wide array of techniques that aimed at enabling computers to process and understand human language. These tasks can be categorized into several broad areas, each addressing different aspects of language processing. Here are some of the key NLP techniques:

1. Text Processing and Preprocessing in NLP

- **Tokenization**: Dividing text into smaller units, such as words or sentences.
- **Stemming and Lemmatization**: Reducing words to their base or root forms.
- **Stopword Removal**: Removing common words (like “and”, “the”, “is”) that may not carry significant meaning.
- **Text Normalization**: Standardizing text, including case normalization, removing punctuation, and correcting spelling errors.

2. Semantic Analysis

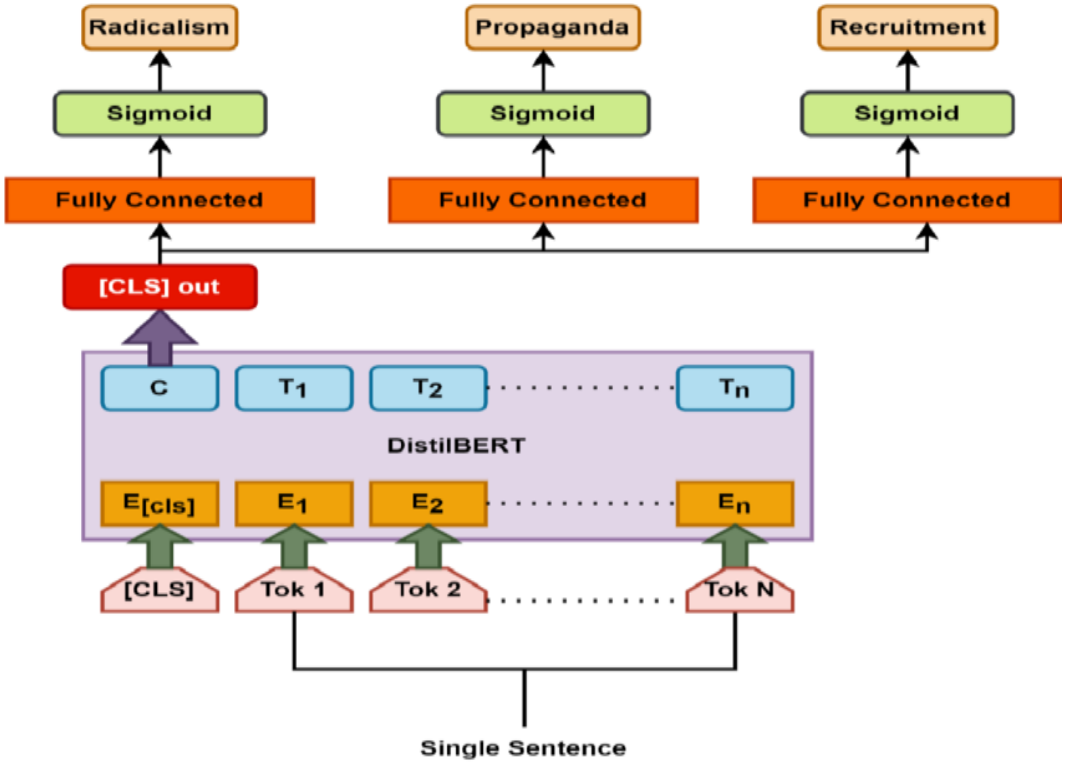
- **Named Entity Recognition (NER)**: Identifying and classifying entities in text, such as names of people, organizations, locations, dates, etc.
- **Word Sense Disambiguation (WSD)**: Determining which meaning of a word is used in a given context.
- **Coreference Resolution**: Identifying when different words refer to the same entity in a text.

3. Text Classification in NLP

- Determining the sentiment or emotional tone expressed in a text (e.g., positive, negative, neutral).
- Identifying topics or themes within a large collection of documents.

1.3.3. Distilled BERT Approach

DistilBERT is a state-of-the-art, lightweight natural language processing (NLP) model developed by Hugging Face as a more efficient alternative to the original BERT (Bidirectional Encoder Representations from Transformers). It is designed to be **40% smaller** and **60% faster** than its predecessor while remarkably retaining approximately **97% of BERT's performance** on key language benchmarks. This efficiency is achieved through a process called **knowledge distillation**, where a smaller "student" model is trained to mimic the behavior and output distributions of the larger "teacher" BERT model, learning from its "dark knowledge" rather than just hard labels. By reducing the number of layers by half and removing specialized components like token-type embeddings, DistilBERT significantly lowers the computational power and memory required for both training and real-time inference.



These characteristics make it a premier choice for **resource-constrained environments**, such as mobile devices, edge computing, and real-time applications like sentiment analysis, question answering, and emotion detection. Ultimately, DistilBERT democratizes access to advanced transformer-based AI by balancing high-level accuracy with the practical agility needed for modern, scalable software deployments.

1.4. AIM AND OBJECTIVE

Aim

The aim of this project is to develop a hybrid system, Opaline Attachment, that enhances email security by detecting and neutralizing malicious attachments using DistilBERT-based analysis and static image rendering, thereby mitigating risks associated with zero-day exploits and sophisticated malware attacks.

Objective

- To implement DistilBERT for detecting and classifying malicious email attachments.
- To convert potentially malicious attachments into static PNG images for safe preview.
- To isolate email attachments in a sandboxed environment to prevent exploitation.
- To provide explicit warnings when accessing original attachments.
- To enhance security against sophisticated and zero-day exploits.

1.5. SCOPE OF THE PROJECT

The scope of the project extends to enhancing email security by mitigating threats posed by malicious attachments. The system leverages DistilBERT for intelligent detection of suspicious files based on content analysis and metadata evaluation. Additionally, it employs a sandboxed environment to convert potentially harmful attachments into static PNG images, ensuring secure previews without executing malicious code. This project is applicable across various domains, including corporate enterprises, government organizations, and individual users, where email security is a critical concern. By integrating advanced AI-driven threat detection and isolation techniques, it provides a proactive defense against zero-day exploits, phishing attempts, and malware-laden attachments. The solution can be further extended with cloud-based deployment, real-time threat intelligence updates, and integration with existing email security frameworks to enhance overall protection and adaptability in evolving cybersecurity landscapes.

CHAPTER 2

SYSTEM ANALYSIS

2.1. EXISTING SYSTEM

The traditional approach to detecting malicious email attachments relies on a combination of manual, signature-based, heuristic, and sandboxing techniques. While these methods have been effective in combating known threats, they struggle against modern, evolving cyber threats, including zero-day exploits, sophisticated phishing campaigns, and polymorphic malware. Below are the key components of traditional email security systems.

- **Malware Detection Systems**

Traditional malware detection systems primarily use signature-based and heuristic-based techniques. Signature-based detection works by comparing email attachments to a database of known malware signatures. While this method is effective against previously identified threats, it fails to detect zero-day malware and polymorphic viruses, which modify their code to evade detection. Heuristic-based detection, on the other hand, analyzes file behavior and code structures to identify potentially malicious attachments. However, this approach often results in false positives or false negatives, reducing overall reliability.

- **Antivirus Software**

Antivirus software is commonly used to scan email attachments for potential threats. It employs a mix of pattern matching and behavioral analysis to detect malware. While antivirus solutions are effective for known threats, they require frequent updates to maintain accuracy. Additionally, some malware exploits vulnerabilities in document formats, such as PDFs and Word documents, making them difficult to detect through traditional antivirus scanning. Polymorphic malware, which changes its structure upon each infection, further weakens antivirus effectiveness.

- **Sandboxing**

Sandboxing is a technique that executes email attachments in a controlled, isolated environment to monitor their behavior before allowing access. This method is effective in detecting zero-day threats and unknown malware by observing suspicious activities such as unauthorized system modifications or network connections. However, sandboxing has several drawbacks, including high resource consumption, detection evasion by advanced malware, and limited integration with email clients, which can slow down email processing.

2.1.1. DISADVANTAGES

- Existing malware detection systems often fail to identify new or unknown threats in email attachments.
- Users may still unknowingly interact with malicious attachments.
- Traditional antivirus software struggles to detect highly-targeted or sophisticated attacks embedded in email attachments.
- Sandboxing solutions can be resource-heavy and are not always integrated seamlessly with email clients.
- Existing systems may not provide real-time protection, allowing malicious attachments to reach users before they are detected.

2.2. PROPOSED SYSTEM

The proposed approach for **CrystallineAttachment** integrates advanced AI-driven threat detection with secure file rendering techniques to enhance email security.

- **Threat Detection Using DistilBERT**

The proposed system leverages **DistilBERT** to analyze the content and metadata of email attachments. By utilizing deep learning-based natural language processing (NLP), the model identifies suspicious patterns, anomalies, and potential indicators of malicious intent. This AI-driven approach enhances detection accuracy, making it effective against evolving threats, including zero-day exploits and sophisticated phishing attacks.

- **Secure File Rendering and Sandboxing**

To eliminate the risk of executing malicious code, the system converts email attachments into **static PNG images** within a **sandboxed environment**. This process ensures that users can preview the content of attachments safely without opening or downloading the original file. By replacing the actual attachment with its rendered version, the system minimizes the chances of malware infections caused by executable scripts embedded in common document formats like PDFs, Word documents, and Excel files.

- **User Awareness and Explicit Warnings**

The system enhances security by providing **explicit warnings** when a potentially malicious attachment is detected. Users are alerted before interacting with any flagged file, reducing the risk of unintentional exposure to harmful content. Additionally, the secure preview feature allows them to verify file contents without compromising system security, improving overall cybersecurity awareness.

2.2.1. ADVANTAGES

- AI-powered threat detection using DistilBERT to analyze email attachment content and metadata.
- Conversion of attachments into secure PNG images to prevent execution of malicious code.
- Sandboxed environment for rendering attachments without compromising system security.
- Explicit warnings to users about potentially harmful attachments before interaction.
- Prevention of zero-day exploits by identifying anomalies and suspicious patterns.
- Seamless integration with existing email systems and security frameworks.
- Enhanced user awareness through safe previewing and educational alerts about risky files.
- Scalable and adaptable architecture for deployment across various organizational environments.

CHATPER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

- **Processor** **:** Minimum Intel i5 or equivalent
- **RAM** **:** 16 GB or higher
- **Storage** **:** 500GB or higher HDD/SSD
- **Network** **:** Stable internet connection

3.2. SOFTWARE REQUIREMENTS

- **Language** : Python 3.7.4 (64-bit or 32-bit)
- **Operating System** : Windows 10 or Linux-based OS
- **Web Browser** : Chrome/Firefox
- **Development Tools** : React JS, WAMP Server
- **ML Framework** : DistilBERT, TensorFlow
- **Database Design** : MySQL

CHAPTER 4

SOFTWARE DESCRIPTION

4.1. PYTHON 3.8

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. It was created by Guido van Rossum during 1985- 1990. Like Perl, Python source code is also available under the GNU General Public License (GPL). This tutorial gives enough understanding on Python programming language.

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time. Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc. The biggest strength of Python is huge collection of standard libraries which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Scientific computing
- Text processing and many more.

Tensor Flow

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.

TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

Pandas

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language. pandas is a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

Pandas is mainly used for data analysis and associated manipulation of tabular data in Data frames. Pandas allows importing data from various file formats such as comma-separated values, JSON, Parquet, SQL database tables or queries, and Microsoft Excel. Pandas allows various data

manipulation operations such as merging, reshaping, selecting, as well as data cleaning, and data wrangling features. The development of pandas introduced into Python many comparable features of working with Data frames that were established in the R programming language. The panda's library is built upon another library NumPy, which is oriented to efficiently working with arrays instead of the features of working on Data frames.

NumPy

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed.

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.

Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK.

Scikit Learn

scikit-learn is a Python module for machine learning built on top of SciPy and is distributed under the 3-Clause BSD license.

Scikit-learn (formerly scikits. learn and also known as sklearn) is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

4.2. MYSQL 5

MySQL is a relational database management system based on the Structured Query Language, which is the popular language for accessing and managing the records in the database. MySQL is open-source and free software under the GNU license. It is supported by Oracle Company. MySQL database that provides for how to manage database and to manipulate data with the help of various SQL queries. These queries are: insert records, update records, delete records, select records, create tables, drop tables, etc. There are also given MySQL interview questions to help you better understand the MySQL database.

MySQL is currently the most popular database management system software used for managing the relational database. It is open-source database software, which is supported by Oracle Company. It is fast, scalable, and easy to use database management system in comparison with Microsoft SQL Server and Oracle Database. It is commonly used in conjunction with PHP scripts for creating powerful and dynamic server-side or web-based enterprise applications. It is developed, marketed, and supported by MySQL AB, a Swedish company, and written in C programming language and C++ programming language. The official pronunciation of MySQL is not the My Sequel; it is My Ess Que Ell. However, you can pronounce it in your way. Many small and big companies use MySQL. MySQL supports many Operating Systems like Windows, Linux, MacOS, etc. with C, C++, and Java languages.

4.3. WAMPSERVER

WampServer is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your database.

WAMPServer is a reliable web development software program that lets you create web apps with MYSQL database and PHP Apache2. With an intuitive interface, the application features numerous functionalities and makes it the preferred choice of developers from around the world. The software is free to use and doesn’t require a payment or subscription.

4.4. BOOTSTRAP 4

Bootstrap is a free and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript framework for developing responsive, mobile-first websites.

It solves many problems which we had once, one of which is the cross-browser compatibility issue. Nowadays, the websites are perfect for all the browsers (IE, Firefox, and Chrome) and for all sizes of screens (Desktop, Tablets, Phablets, and Phones). All thanks to Bootstrap developers -Mark Otto and Jacob Thornton of Twitter, though it was later declared to be an open-source project.

Easy to use: Anybody with just basic knowledge of HTML and CSS can start using Bootstrap

Responsive features: Bootstrap's responsive CSS adjusts to phones, tablets, and desktops

Mobile-first approach: In Bootstrap, mobile-first styles are part of the core framework

Browser compatibility: Bootstrap 4 is compatible with all modern browsers (Chrome, Firefox, Internet Explorer 10+, Edge, Safari, and Opera)

4.5. FLASK

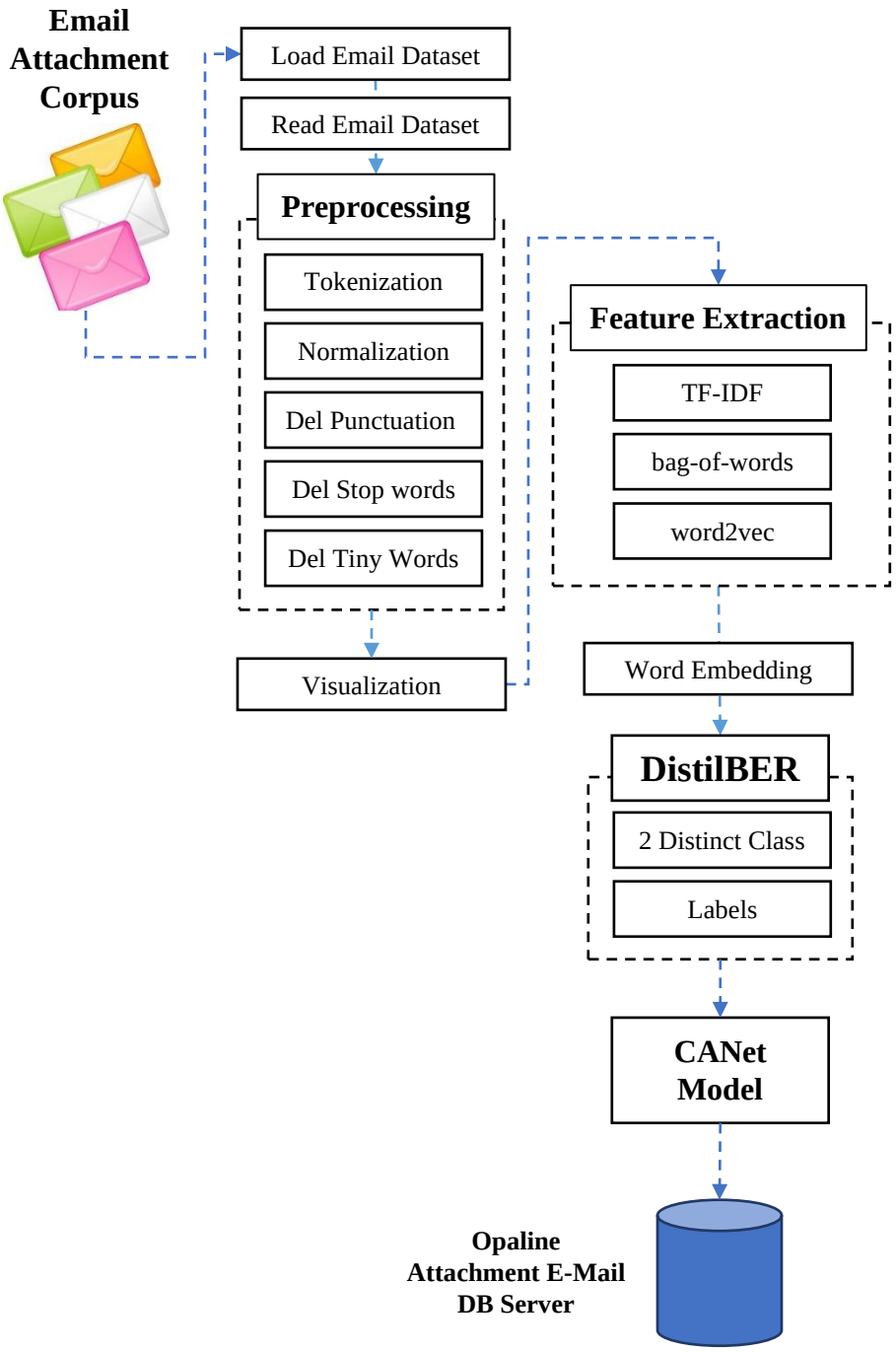
Flask is a web framework. This means flask provides you with tools, libraries and technologies that allow you to build a web application. This web application can be some web pages, a blog, a wiki or go as big as a web-based calendar application or a commercial website.

Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have formed a validation support. Instead, Flask supports the extensions to add such functionality to the application. Although Flask is rather young compared to most Python frameworks, it holds a great promise and has already gained popularity among Python web developers. Let's take a closer look into Flask, so-called "micro" framework for Python. Flask is part of the categories of the micro-framework. Micro-framework are normally framework with little to no dependencies to external libraries. This has pros and cons. Pros would be that the framework is light, there are little dependency to update and watch for security bugs, cons is that some time you will have to do more work by yourself or increase yourself the list of dependencies by adding plugins.

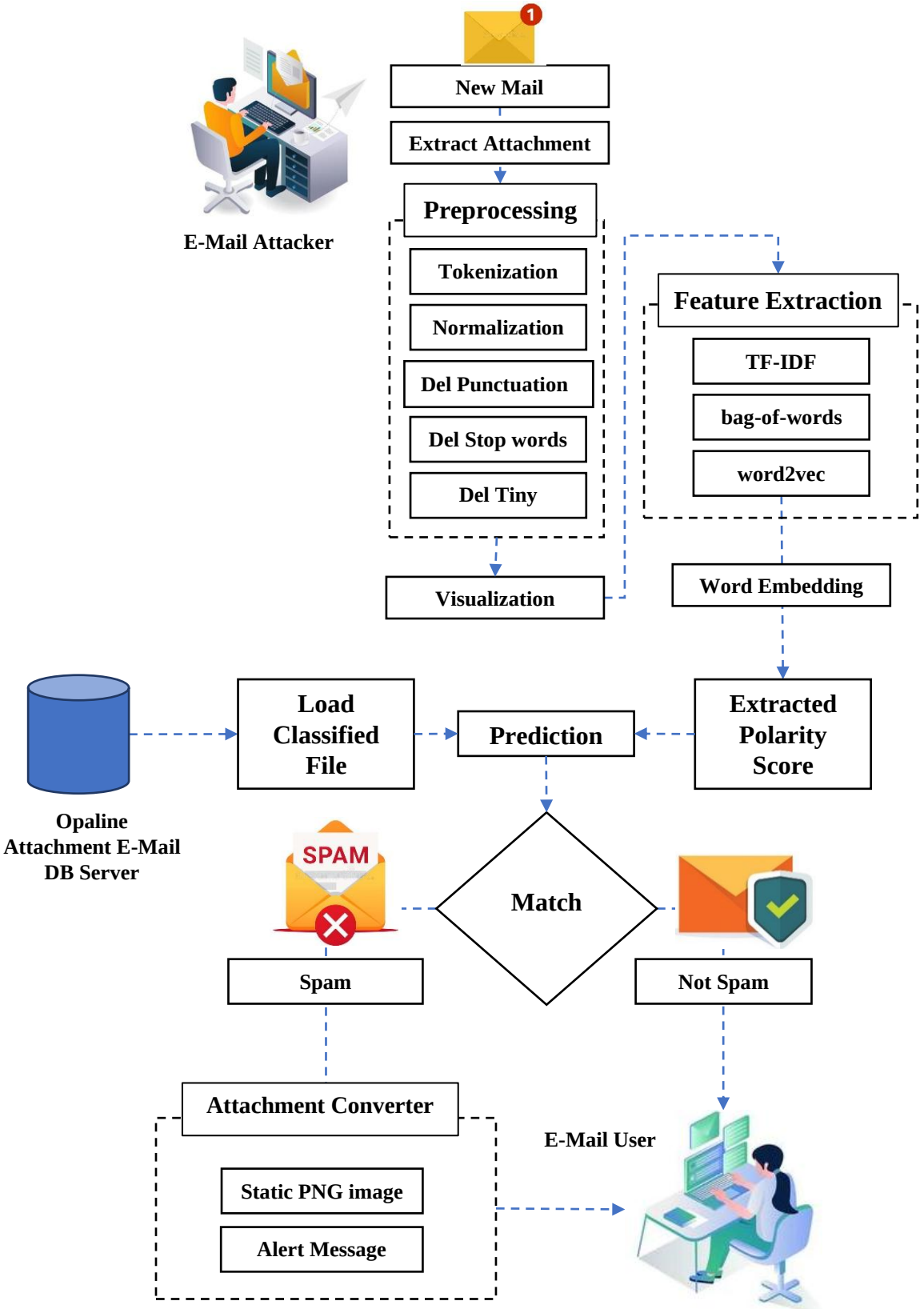
CHAPTER 5

SYSTEM DESIGN

5.1. SYSTEM ARCHITECTURE

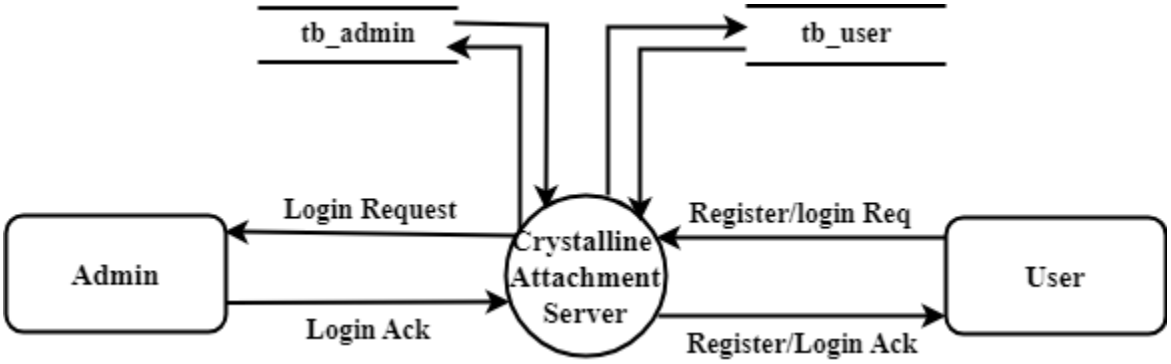


Testing Phase

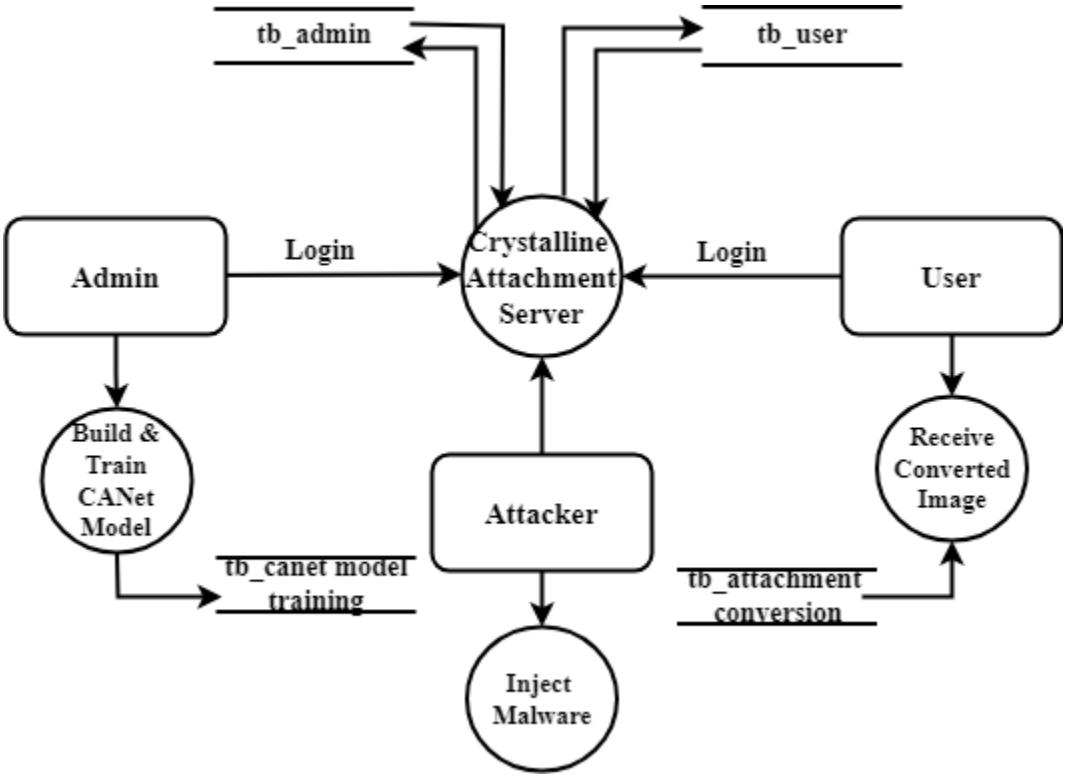


5.2. DATA FLOW DIAGRAM

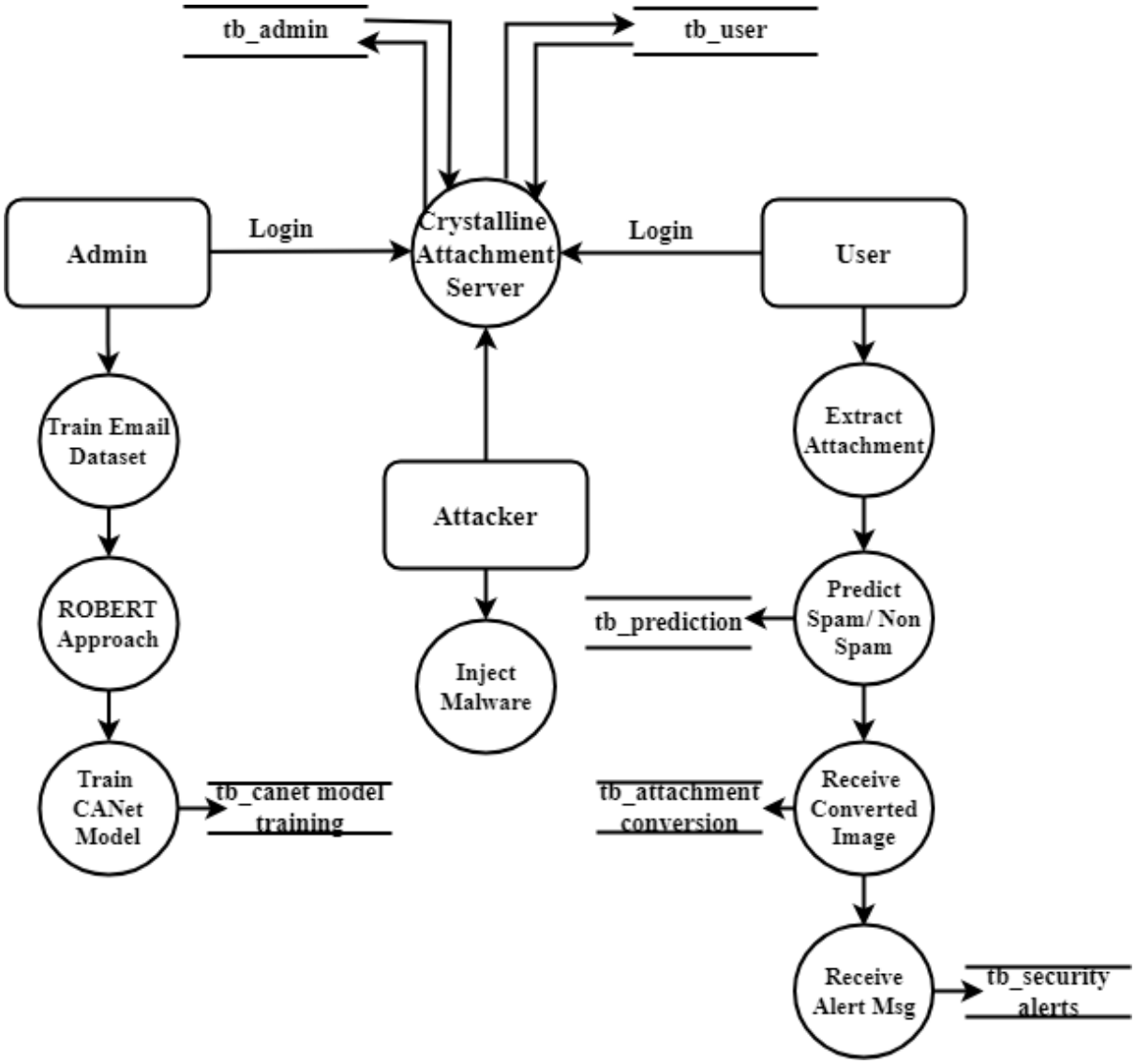
LEVEL 0



LEVEL 1

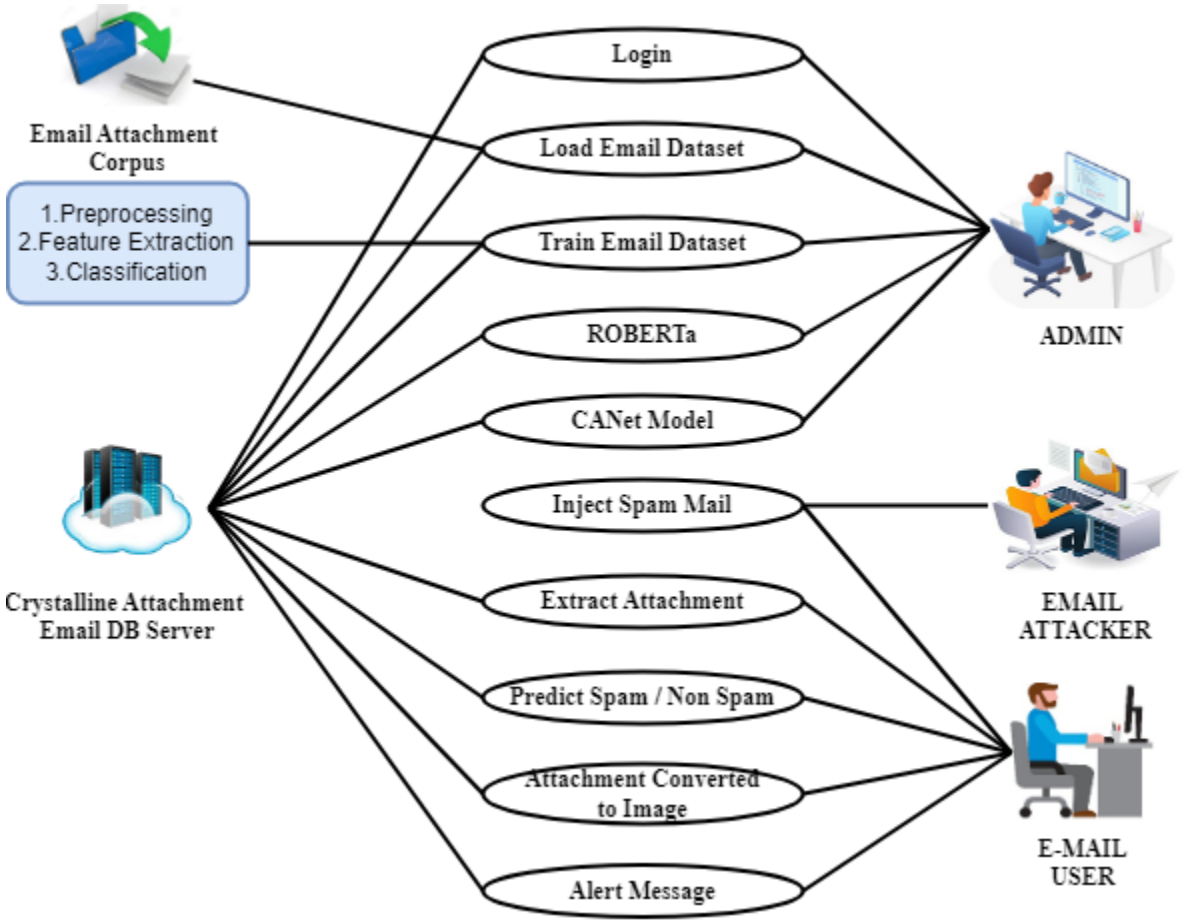

tb_admin
tb_user
Admin
Login
Crystalline Attachment Server
Login
User
Build & Train CANet Model
Attacker
Receive Converted Image
tb_canet model training
tb_attachment conversion
Inject Malware

LEVEL 2

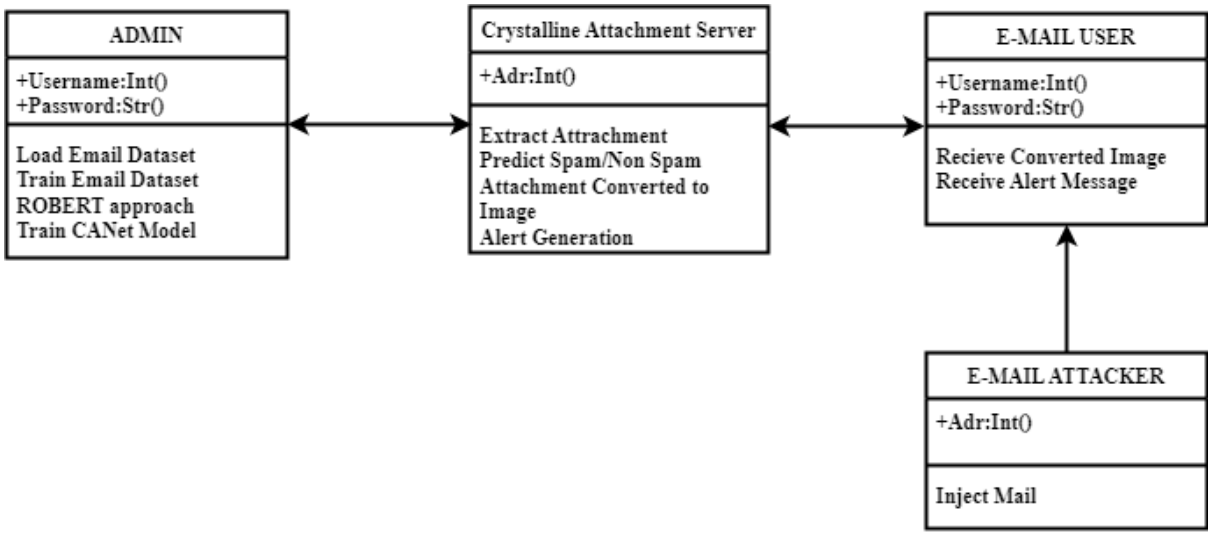

tb_admin
tb_user
Admin
Login
Crystalline Attachment Server
Login
User
Train Email Dataset
Attacker
Extract Attachment
ROBERT Approach
tb_prediction
Predict Spam/ Non Spam
Inject Malware
Train CANet Model
tb_canet model training
tb_attachment conversion
Receive Converted Image
Receive Alert Msg
tb_security alerts

5.3. UML DIAGRAM

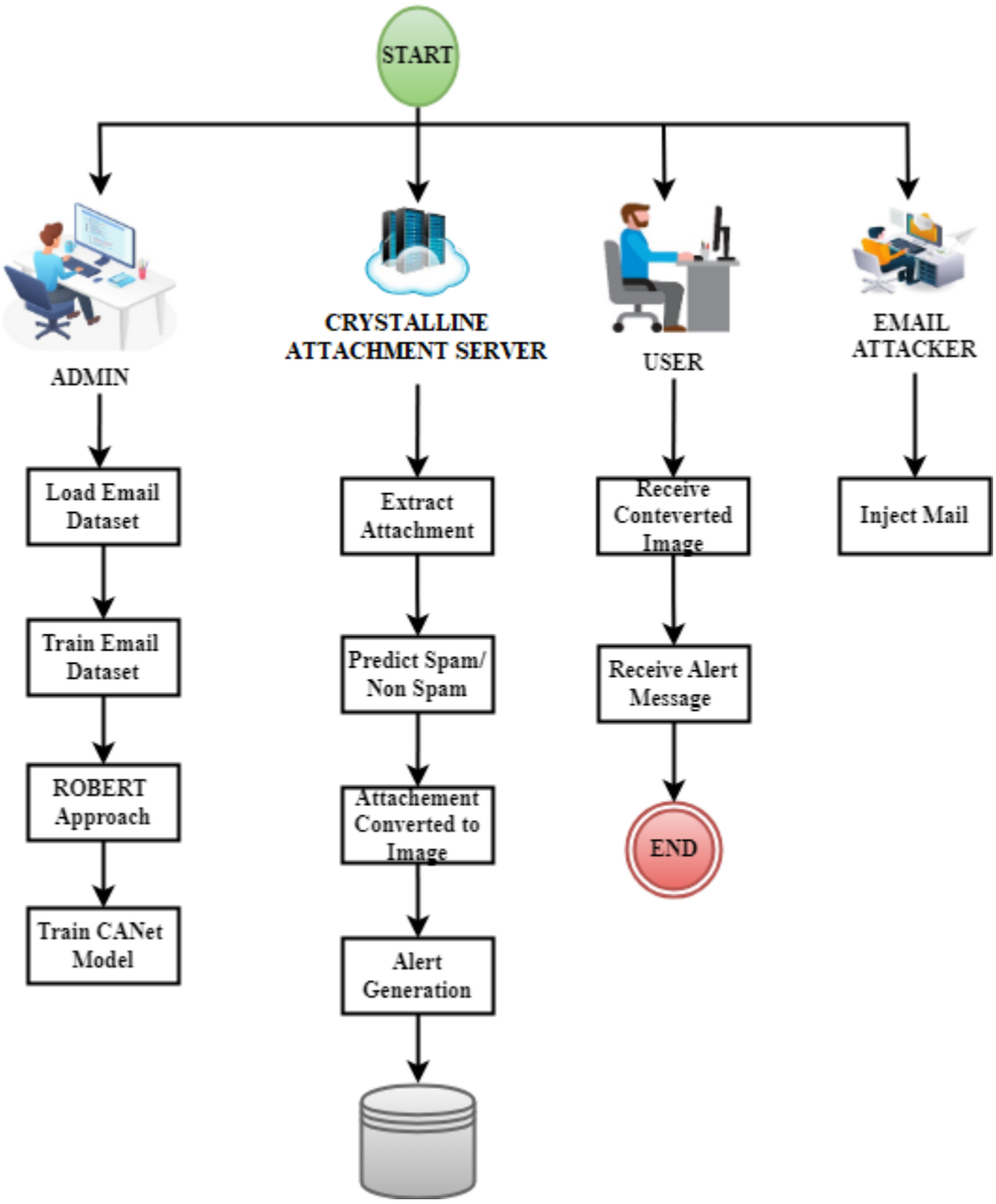
5.3.1. USE CASE



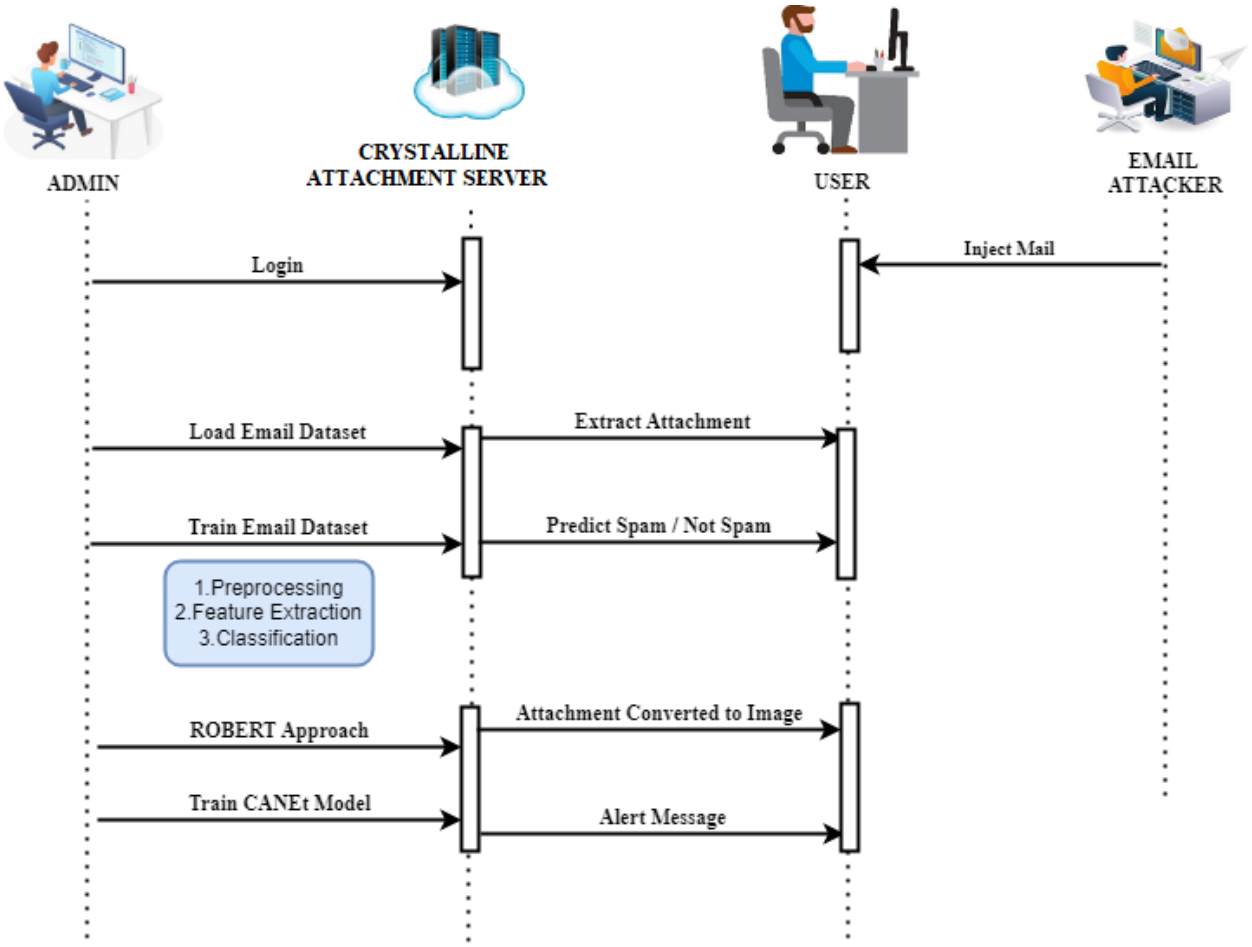
5.3.2. CLASS DIAGRAM



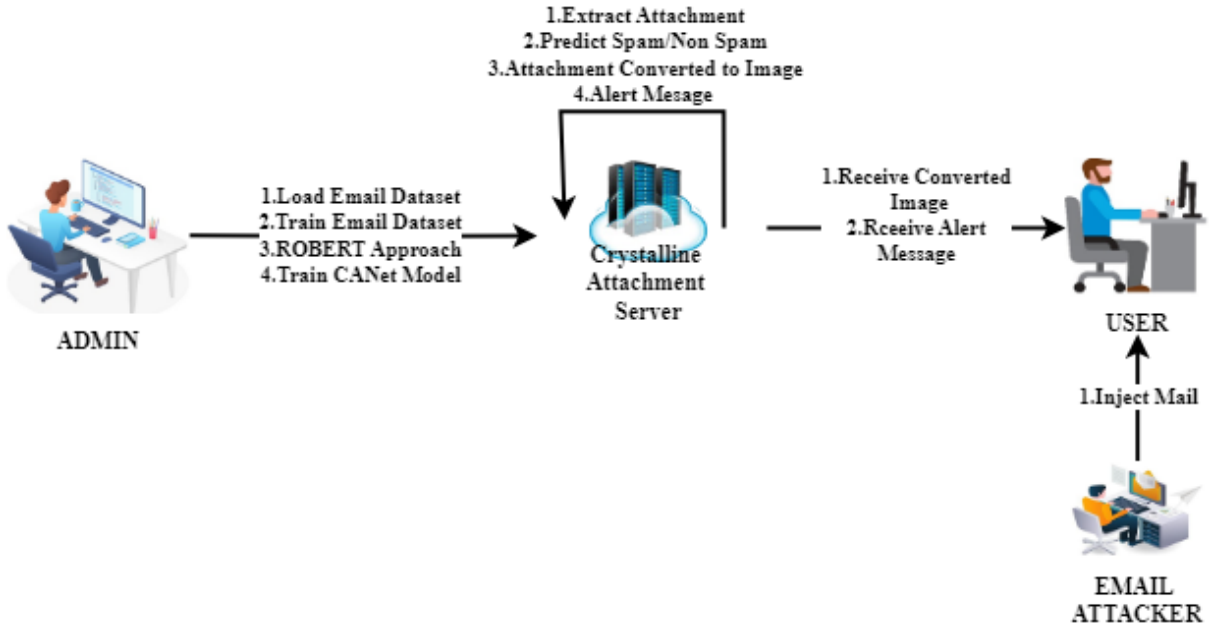
5.3.3. ACTIVITY DIAGRAM



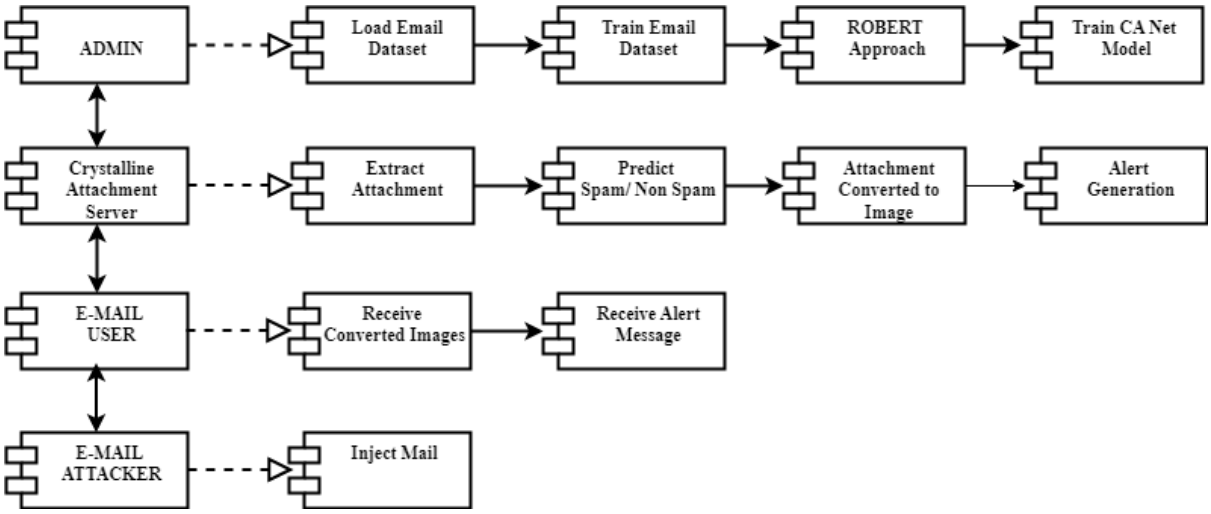
5.3.4. SEQUENCE DIAGRAM



5.3.5. COLLABORATION DIAGRAM



5.3.6. COMPONENT DIAGRAM



5.4. TABLE DESIGN

1. User Table

Column Name	Data Type	Description
User_Id	Int (Primary Key)	Unique Identifier For Each User.
First_Name	Varchar(100)	User's First Name.
Last_Name	Varchar(100)	User's Last Name.
Email	Varchar(255)	User's Email Address, Used For Login And Notifications.
Password	Varchar(255)	Encrypted Password For User Authentication.
Role	Varchar(50)	User's Role (Admin/User). Defines User Privileges.
Created_At	Timestamp	Timestamp Of When The User Account Was Created.
Updated_At	Timestamp	Timestamp Of When The User Account Was Last Updated.

2. Admin Table

Column Name	Data Type	Description
Admin_Id	Int (Primary Key)	Unique Identifier For Each Admin.
First_Name	Varchar(100)	Admin's First Name.
Last_Name	Varchar(100)	Admin's Last Name.
Email	Varchar(255)	Admin's Email Address.
Password	Varchar(255)	Encrypted Password For Admin Authentication.
Created_At	Timestamp	Timestamp Of When The Admin Account Was Created.
Updated_At	Timestamp	Timestamp Of When The Admin Account Was Last Updated.

3. Email Configuration Table

Column Name	Data Type	Description
Config_Id	Int (Primary Key)	Unique Identifier For Email Configuration.
User_Id	Int (Foreign Key)	Reference To The User Who Owns The Email Configuration.
Email_Address	Varchar(255)	Email Address For The Configuration.
Smtp_Server	Varchar(255)	Smtp Server Address For Sending Emails.
Smtp_Port	Int	Port For The Smtp Server.
Imap_Server	Varchar(255)	Imap Server Address For Receiving Emails.
Imap_Port	Int	Port For The Imap Server.
Created_At	Timestamp	Timestamp Of When The Email Configuration Was Added.
Updated_At	Timestamp	Timestamp Of When The Email Configuration Was Last Updated.

4. Canet Model Training Data Table

Column Name	Data Type	Description
Training_Id	Int (Primary Key)	Unique Identifier For Each Training Dataset.
Dataset_Name	Varchar(255)	Name Of The Dataset Used For Training.
Feature_Set	Text	List Of Features Used In Training.
Training_Data	Text	Actual Training Data (Could Be Stored In A Serialized Form).
Created_At	Timestamp	Timestamp Of When The Training Data Was Added.

5. Prediction Table

Column Name	Data Type	Description
Prediction_Id	Int (Primary Key)	Unique Identifier For Each Prediction.
User_Id	Int (Foreign Key)	Reference To The User Who Made The Prediction Request.
Email_Id	Int (Foreign Key)	Reference To The Email Being Predicted.
Prediction_Result	Varchar(255)	The Result Of The Prediction (E.G., Spam/Ham).
Prediction_Timestamp	Timestamp	Timestamp When The Prediction Was Made.
Model_Version	Varchar(50)	Version Of The Canet Model Used For Prediction.

6. Attachment Conversion Table

Column Name	Data Type	Description
Conversion_Id	Int (Primary Key)	Unique Identifier For Each Attachment Conversion.
Email_Id	Int (Foreign Key)	Reference To The Email With The Attachment.
Original_Attachment	Varchar(255)	Path To The Original Attachment.
Converted_Attachment	Varchar(255)	Path To The Converted Png Image Attachment.
Conversion_Status	Varchar(50)	Status Of The Conversion (E.G., Successful, Failed).
Conversion_Timestamp	Timestamp	Timestamp When The Conversion Was Performed.

7. Security Alerts Table

Column Name	Data Type	Description
Alert_Id	Int (Primary Key)	Unique Identifier For Each Security Alert.
User_Id	Int (Foreign Key)	Reference To The User Who Received The Alert.
Email_Id	Int (Foreign Key)	Reference To The Email Associated With The Alert.
Alert_Message	Text	The Content Of The Security Alert Message.
Alert_Type	Varchar(50)	Type Of Alert (E.G., Malicious Attachment, Suspicious Activity).
Alert_Timestamp	Timestamp	Timestamp Of When The Alert Was Triggered.
Status	Varchar(50)	Status Of The Alert (E.G., Resolved, Pending).

CHAPTER 6

SYSTEM TESTING

6.1. SOFTWARE TESTING

Software testing is a crucial process that ensures the Opaline Attachment E-Mail Server Web App functions effectively and securely. The testing phase begins early in development and continues until deployment, allowing the identification and resolution of potential defects. Given the dynamic nature of cybersecurity threats, rigorous testing is essential to ensure the system remains reliable, secure, and efficient in detecting and handling malicious attachments.

6.1.1. Software Testing Process

The software testing process follows a structured four-step approach to ensure thorough evaluation and reliability. The first step is Planning, where the testing scope is defined, key functionalities are identified, and test cases are outlined. The second step, Preparing, involves setting up the test environment, selecting test tools, and designing test cases. The third step, Executing, includes running test cases, recording results, and identifying defects. Finally, the Reporting stage involves documenting test outcomes, analyzing issues, and providing feedback for necessary improvements.

- **Functional Testing**

Functional testing ensures that the system operates according to its intended specifications. Unit Testing is conducted on individual components such as email parsing and malware detection modules to verify their correctness. Integration Testing ensures smooth communication between different modules, such as attachment scanning and security alerts. System Testing evaluates the complete functionality of the application, including email filtering and threat detection. Acceptance Testing is carried out by end-users to validate that the application meets their needs before deployment. Interface Testing ensures seamless interaction between the email server, malware detection engine, and user dashboard. Regression Testing is performed after updates to ensure that new changes do not introduce new defects.

- **Non-Functional Testing**

Non-functional testing assesses the system's performance, security, and usability. Security Testing is performed to detect vulnerabilities and prevent cyber threats like malware and phishing attacks. Performance Testing evaluates system responsiveness and stability under different loads. Load

Testing simulates multiple users accessing the system simultaneously to check its efficiency. Stress Testing pushes the system beyond normal usage conditions to determine its breaking point. Usability Testing ensures the interface is user-friendly and accessible. Compatibility Testing verifies that the application works seamlessly across different operating systems, browsers, and devices. Recovery Testing assesses how well the system recovers from failures such as server crashes or data corruption.

6.2. TEST CASE

Authentication & User Management

1. **Test Case ID:** TC001
    - **Input:** Admin attempts to log in with valid credentials.
    - **Expected Result:** Successful login, granting access to administrative functionalities.
    - **Actual Result:** Admin successfully logged in, access granted.
    - **Status:** Pass
2. **Test Case ID:** TC002
    - **Input:** User attempts to log in with incorrect credentials.
    - **Expected Result:** Login fails, displaying an error message.
    - **Actual Result:** Error message displayed, login unsuccessful.
    - **Status:** Pass
3. **Test Case ID:** TC003
    - **Input:** New user registers with valid details.
    - **Expected Result:** Account successfully created, user can log in.
    - **Actual Result:** User account created, login successful.
    - **Status:** Pass
4. **Test Case ID:** TC004
    - **Input:** User tries to register with an existing email ID.
    - **Expected Result:** Registration fails, system shows "Email already exists" error.
    - **Actual Result:** Error message displayed, registration blocked.
    - **Status:** Pass

Model Training

5. **Test Case ID:** TC005
    - **Input:** Admin uploads a diverse dataset for training the CANet model.
    - **Expected Result:** Dataset uploaded successfully, ready for model training.
    - **Actual Result:** Dataset uploaded successfully, system ready for training.
    - **Status:** Pass
6. **Test Case ID:** TC006
    - **Input:** Admin starts the CANet model training process.

- **Expected Result:** Model training starts and completes without errors.
- **Actual Result:** Model trained successfully with logs generated.
- **Status:** Pass

7. **Test Case ID:** TC007
    - **Input:** Admin uploads a corrupted dataset file.
    - **Expected Result:** System rejects the file and displays an error message.
    - **Actual Result:** System detected corruption and rejected the file.
    - **Status:** Pass

Malicious E-Mail Attachment Detection

8. **Test Case ID:** TC008
    - **Input:** User receives an email with a malicious attachment.
    - **Expected Result:** System detects and flags the attachment as malicious.
    - **Actual Result:** Attachment correctly flagged as malicious.
    - **Status:** Pass
9. **Test Case ID:** TC009
    - **Input:** User receives an email with a safe attachment.
    - **Expected Result:** System processes the email without flagging the attachment.
    - **Actual Result:** Attachment remains unflagged.
    - **Status:** Pass
10. **Test Case ID:** TC010
    - **Input:** Email with a malicious attachment is opened by the user.
    - **Expected Result:** System blocks direct opening and provides a warning.
    - **Actual Result:** Warning displayed, preventing direct access.
    - **Status:** Pass

Attachment Conversion & Security Handling

11. **Test Case ID:** TC011
    - **Input:** Malicious email attachment is processed by the system.
    - **Expected Result:** System converts the attachment into a static image format.
    - **Actual Result:** Attachment successfully converted to a PNG image.
    - **Status:** Pass

12. **Test Case ID:** TC012
    - **Input:** Safe email attachment is processed.
    - **Expected Result:** System allows the file to be downloaded normally.
    - **Actual Result:** File downloaded without conversion.
    - **Status:** Pass
13. **Test Case ID:** TC013
    - **Input:** User tries to open a flagged malicious attachment.
    - **Expected Result:** System blocks access and provides an alert message.
    - **Actual Result:** Warning displayed, access denied.
    - **Status:** Pass
14. **Test Case ID:** TC014
    - **Input:** User receives an email alert about a detected malicious attachment.
    - **Expected Result:** Alert message sent to the user's registered email ID.
    - **Actual Result:** Alert received by the user.
    - **Status:** Pass

6.3. TEST REPORT

Introduction

The purpose of this test report is to document the testing activities performed on the project. The application aims to detect and handle malicious email attachments using machine learning-based CANet model, providing a secure email environment. The testing process includes functionality validation, security verification, and performance assessment.

Test Objective

The primary objectives of testing are:

- To ensure the authentication and user management functionalities work correctly.
- To validate the correct implementation of dataset uploading and CANet model training.
- To confirm the malicious email attachment detection and classification.
- To check the accuracy of the attachment conversion mechanism.
- To verify system security alerts and warnings for malicious attachments.
- To evaluate system stability and performance under real-time conditions.

Test Scope

The testing scope includes:

- **Functional Testing:** Validation of login, dataset upload, model training, email processing, and attachment handling.
- **Security Testing:** Identification and handling of malicious attachments, warning alerts, and access restrictions.
- **Performance Testing:** System response time and stability under different workloads.
- **Usability Testing:** User experience in configuring email detection and receiving alerts.

Test Environment

- **Hardware:**
    - Processor: Intel i5/i7, 3.0 GHz or higher
    - RAM: 8GB or higher
    - Storage: 256GB SSD or higher
- **Software:**
    - **Front-end:** React JS
    - **Back-end:** Python Flask
    - **Database:** MySQL

- **Local Server:** Wampserver
- **Model:** DistilBERT-based CANet

Test Conclusion

The project has successfully passed all functional, security, and performance tests. The authentication system, dataset upload, model training, email attachment processing, and security alert mechanisms function as expected. Malicious email attachments are accurately detected and converted to ensure safe viewing. The system provides a secure and user-friendly experience for both admins and email account holders. Further improvements may focus on optimizing model training speed and enhancing real-time attachment analysis.

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1. PROJECT DESCRIPTION

The Opaline Attachment E-Mail Server Web App is an advanced email security solution designed to detect and neutralize malicious attachments in emails. This system leverages machine learning and natural language processing (NLP) techniques to analyze email attachments in real-time, ensuring user protection against malware, phishing attempts, and document-based exploits.

The project is developed using Python as the core backend technology, with React JS for an interactive and user-friendly front-end. MySQL is used for storing user data, email metadata, and processed attachments, while WampServer facilitates local development and testing. A key feature of this system is the CANet model, built using DistilBERT, which classifies attachments as either safe or malicious based on extracted features. The system follows a structured workflow, including dataset import, preprocessing, feature extraction, classification, model training, and deployment. Once deployed, the model scans email attachments, detects threats, and flags potentially harmful files. To further enhance security, the system incorporates an attachment conversion mechanism that transforms suspicious files into static images (e.g., PNG format), neutralizing any embedded malicious elements while preserving content accessibility. Additionally, real-time security alerts notify users about detected threats, preventing accidental execution of harmful files. The Attachment E-Mail Server Web App provides an intelligent, automated, and proactive approach to email security, ensuring a safer communication environment for users by mitigating cyber threats effectively.

7.2. MODULES DESCRIPTION

1. Opaline Attachment E-Mail Server Web App

The Opaline Attachment E-Mail Server Web App is an advanced email security solution designed to detect and neutralize malicious attachments. The system is built using Python as the core backend technology, leveraging its powerful data processing and machine learning capabilities to enhance security measures. The front-end interface is developed using React JS, ensuring a dynamic, interactive, and user-friendly experience. React JS enables fast rendering and efficient component management, making the UI responsive and seamless for users. The system employs MySQL for database management, which is responsible for storing and organizing critical data such as user account information, including registration details, authentication credentials, and user roles. It also manages email metadata, including sender and receiver information, timestamps, and email content, as well as processed attachments, consisting of original files, their security status, and converted formats to ensure safe access. To support development and deployment, the system utilizes Wampserver as a local development environment. Wampserver provides an all-in-one hosting solution, integrating Apache, MySQL, and PHPMyAdmin to facilitate efficient testing and deployment of the application. This ensures seamless integration and smooth functionality of the email security system.

2. System User Roles and Operations

The system is designed to cater to two primary **user roles**, each with specific functionalities:

2.1. Admin

The **Admin** holds full control over the system and is responsible for:

- **Login Authentication**: Securing system access through user authentication mechanisms.
- **Dataset Management**: Uploading and managing email attachment datasets used for model training.
- **CANet Model Training**: Building, training, and fine-tuning the **CANet** model to detect malicious attachments effectively.
- **User Account Management**: Handling user registration, login authentication, and setting user permissions.

2.2. E-Mail Account Holder/User

The **end-users** (email account holders) interact with the system to:

- **Register and log in** to access the email attachment security features.
- **Configure their email accounts** for real-time attachment monitoring.
- **View converted attachments** directly in their email client to ensure safe access.
- **Receive alerts and warnings** about potentially malicious email attachments before opening them.

3. CANet Model: Build and Train

The **CANet model** is at the core of the system, using advanced **machine learning techniques** to detect malicious attachments in emails. The training process involves the following steps:

3.1. Import Dataset

The system begins by importing a dataset containing email attachments and metadata, ensuring a diverse mix of both safe and malicious email content. This dataset includes legitimate email attachments such as standard documents, spreadsheets, and images that do not pose any security threats. Additionally, it incorporates malicious attachments, including phishing PDFs, malware-infected Word files, and harmful scripts designed to exploit system vulnerabilities. By maintaining a balanced dataset, the model can effectively learn to differentiate between safe and harmful attachments, improving the accuracy of threat detection.

3.2. Preprocessing

To enhance analytical accuracy, the system applies Natural Language Processing (NLP) techniques to refine email content and prepare it for analysis. This involves cleaning the text by removing unnecessary elements such as stop words, special characters, and redundant metadata. Formatting is standardized to ensure uniformity across all emails, enabling consistent processing. Structured data is extracted from raw email content, allowing for meaningful analysis that aids in distinguishing legitimate emails from potentially harmful ones.

3.3. Feature Extraction

The system utilizes Term Frequency-Inverse Document Frequency (TF-IDF) to extract significant textual patterns from email attachments. This method helps in identifying unusual word distributions, suspicious terms frequently found in phishing emails, and anomalies in email structure that may indicate spam or malware presence. By focusing on these features, the system can detect subtle linguistic patterns that are commonly associated with malicious email content.

3.4. Classification

To ensure precise identification of malicious attachments, the system employs DistilBERT a state-of-the-art deep learning model specialized in Natural Language Processing. DistilBERT classifies emails into two categories: safe, which includes non-malicious attachments, and malicious, which encompasses harmful files containing malware, phishing attempts, or exploits. The deep learning model enables the system to adaptively learn complex patterns, making it more effective in detecting evolving cyber threats.

3.5. Build and Train Model

The CANet model undergoes a rigorous training process using the preprocessed datasets and extracted features. Supervised learning techniques are applied to refine classification accuracy, minimizing errors in detecting threats. The model's parameters are optimized using deep learning methods to reduce false positives and false negatives, ensuring reliable results. Additionally, the model is continuously fine-tuned with new data, enabling it to stay updated with the latest cyber threats and evolving attack patterns.

3.6. Deploy Model

Once the CANet model is trained and validated, it is integrated into the Opaline Attachment E-Mail Server Web App for real-time detection of malicious email attachments. This deployment ensures that every incoming email attachment is automatically analyzed, classified, and flagged if detected as a potential threat. By seamlessly integrating the model within the email server, the system provides users with an added layer of security, protecting them from phishing attempts, malware infections, and other cyber threats.

4. Malicious E-Mail Attachment Detection

Once an email is received, the system analyzes its attachments in real-time to detect potential security threats. The process includes:

4.1. Email Reception

The system begins by processing incoming emails, automatically scanning their attachments for any signs of suspicious or potentially harmful content. Every email that enters the system undergoes a preliminary security check, ensuring that no malicious files reach the user's inbox without being analyzed. This proactive approach helps in identifying threats before they can cause any harm.

4.2. Attachment Processing

Once an email is received, its attachments are extracted and subjected to a detailed analysis to detect potentially harmful elements. The system inspects the file headers, metadata, and embedded scripts for any anomalies that may indicate the presence of malware, phishing attempts, or unauthorized modifications. This step ensures that even deeply hidden threats within an attachment can be identified before they execute on the user's device.

4.3. Malicious Content Detection

The CANet model then processes the extracted data to detect harmful content. Using advanced machine learning techniques, it examines the behavioral patterns within the attachment, assessing whether the file exhibits characteristics commonly associated with malware or phishing attacks. The model compares the file's features against a database of known malicious indicators, classifying it as either safe or harmful based on its findings. This deep-learning approach enhances detection accuracy, reducing false positives while effectively identifying new and emerging threats.

4.4. Flagging Attachments

If an attachment is classified as malicious, the system takes immediate action to prevent potential damage. The email is flagged as a security threat in the user's inbox, ensuring that the recipient is alerted to the risk. Additionally, access to the original file is restricted to prevent accidental execution of harmful code. The user is then notified about the detected security risk, allowing them to take necessary precautions. By implementing these protective measures, the system enhances overall email security, minimizing the risk of cyberattacks and data breaches.

5. Attachment Conversion

To ensure the safety of email recipients, the system will convert potentially malicious attachments into static images. The system will automatically convert suspicious email attachments (such as PDFs, Word documents, etc.) into static PNG images, which will neutralize any harmful code or active elements within the file. This ensures that users can safely view the contents of the attachment without executing any malicious code.

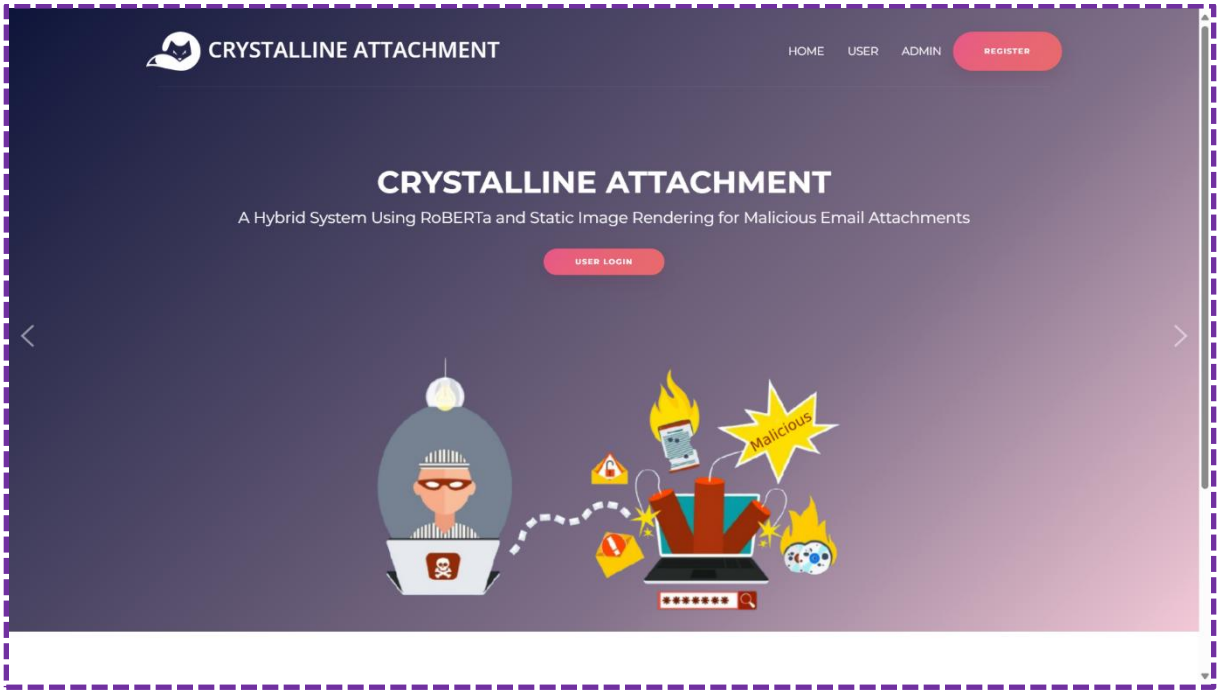
6. Security Handling

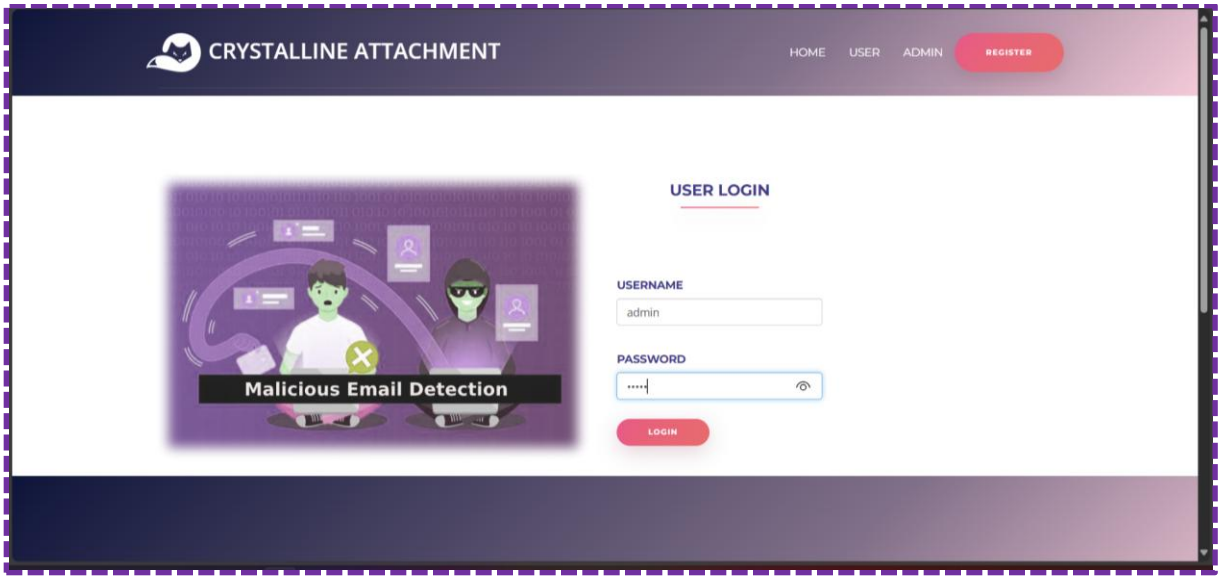
Security is a key aspect of the system, providing users with alerts and warnings about potential threats.When a malicious attachment is detected, the system will issue a warning to the user, notifying them of the risks associated with opening the original file. Users will be informed about the security status of attachments before they decide to interact with them, reducing the risk of compromising their system.

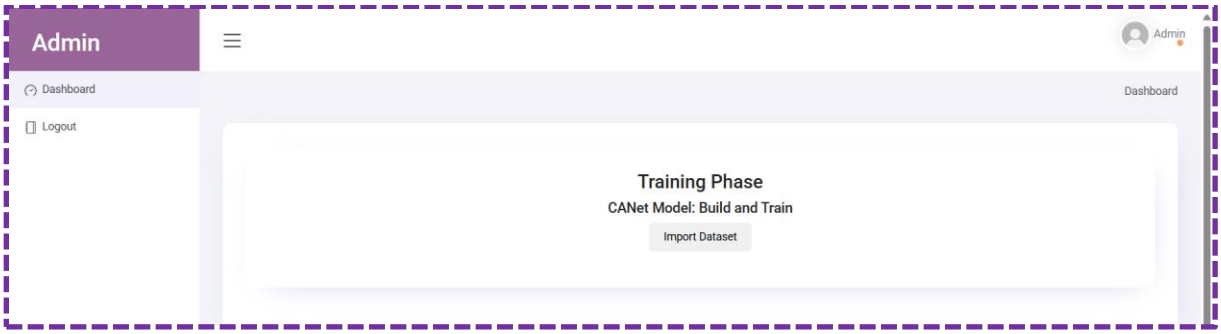
CHAPTER 8

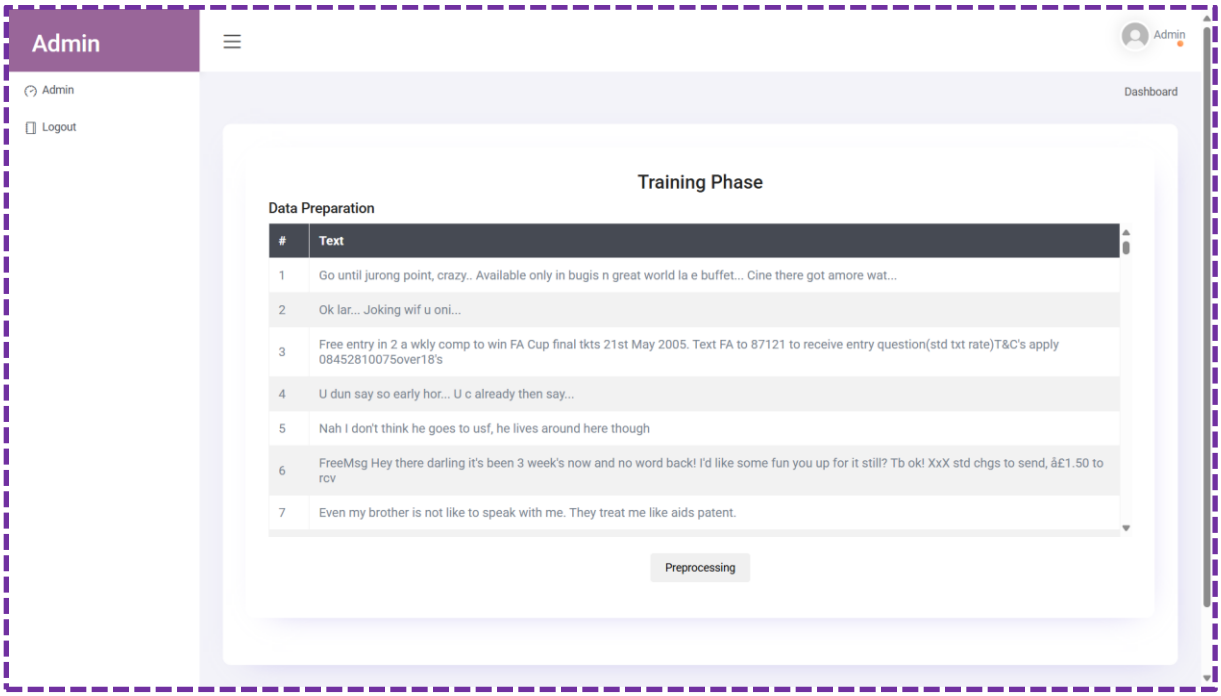
APPENDICES

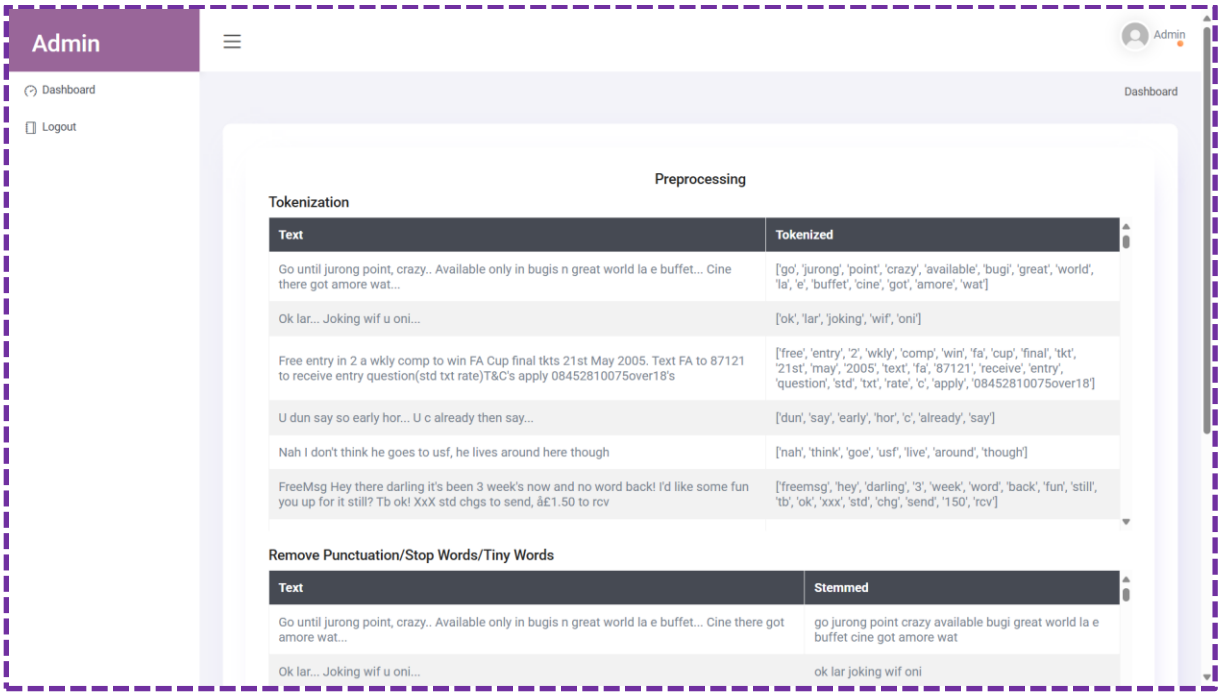
8.1. SCREENSHOTS



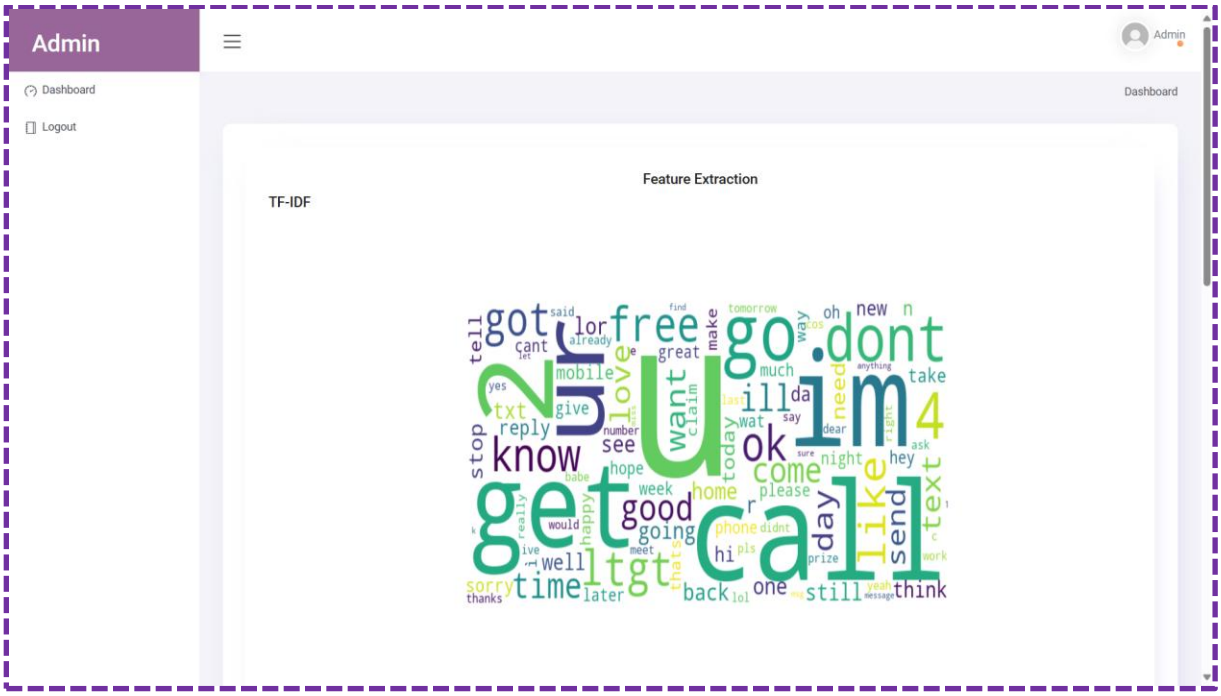


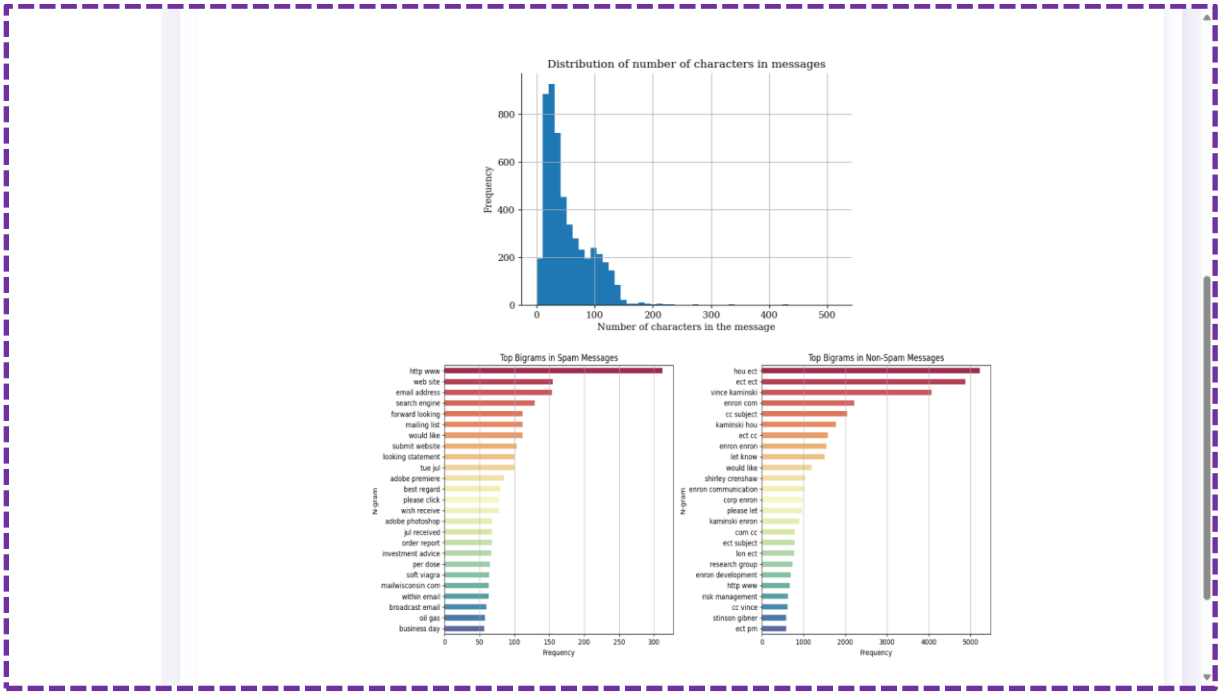

Admin
Admin
Dashboard
Logout
Dashboard
Training Phase
CANet Model: Build and Train
Import Dataset


Admin
Admin
Admin
Dashboard
Logout
Dashboard
Training Phase
Data Preparation
Text
1 Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
2 Ok lar... Joking wif u oni...
3 Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
4 U dun say so early hor... U c already then say...
5 Nah I don't think he goes to usf, he lives around here though
6 FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, â£1.50 to rcv
7 Even my brother is not like to speak with me. They treat me like aids patent.
Preprocessing

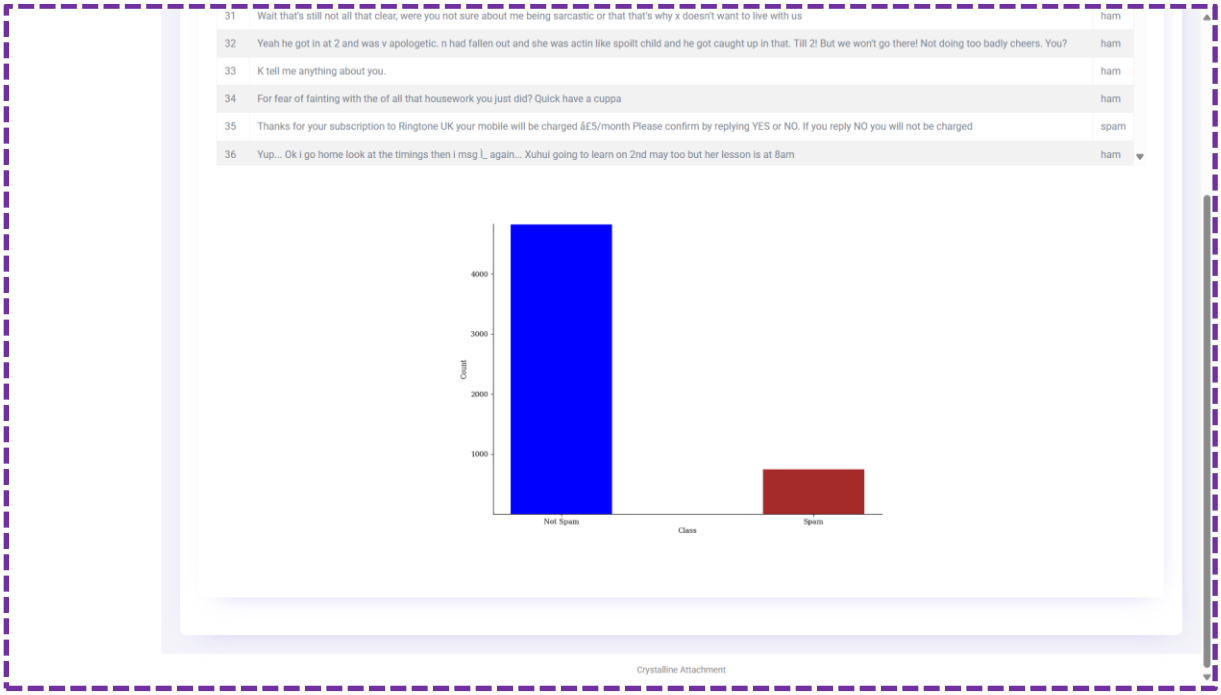

Admin
Admin
Dashboard
Logout
Dashboard
Preprocessing
Tokenization
Text
Tokenized
Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
['go', 'jurong', 'point', 'crazy', 'available', 'bugi', 'great', 'world', 'la', 'e', 'buffet', 'cine', 'got', 'amore', 'wat']
Ok lar... Joking wif u oni...
['ok', 'lar', 'joking', 'wif', 'oni']
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
['free', 'entry', '2', 'wkly', 'comp', 'win', 'fa', 'cup', 'final', 'tkt', '21st', 'may', '2005', 'text', 'fa', '87121', 'receive', 'entry', 'question', 'std', 'txt', 'rate', 'c', 'apply', '08452810075over18']
U dun say so early hor... U c already then say...
['dun', 'say', 'early', 'hor', 'c', 'already', 'say']
Nah I don't think he goes to usf, he lives around here though
['nah', 'think', 'goe', 'usf', 'live', 'around', 'though']
FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, â£1.50 to rcv
['freemsg', 'hey', 'darling', '3', 'week', 'word', 'back', 'fun', 'still', 'tb', 'ok', 'xxx', 'std', 'chg', 'send', '150', 'rcv']
Remove Punctuation/Stop Words/Tiny Words
Text
Stemmed
Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
go jurong point crazy available bugi great world la e buffet cine got amore wat
Ok lar... Joking wif u oni...
ok lar joking wif oni

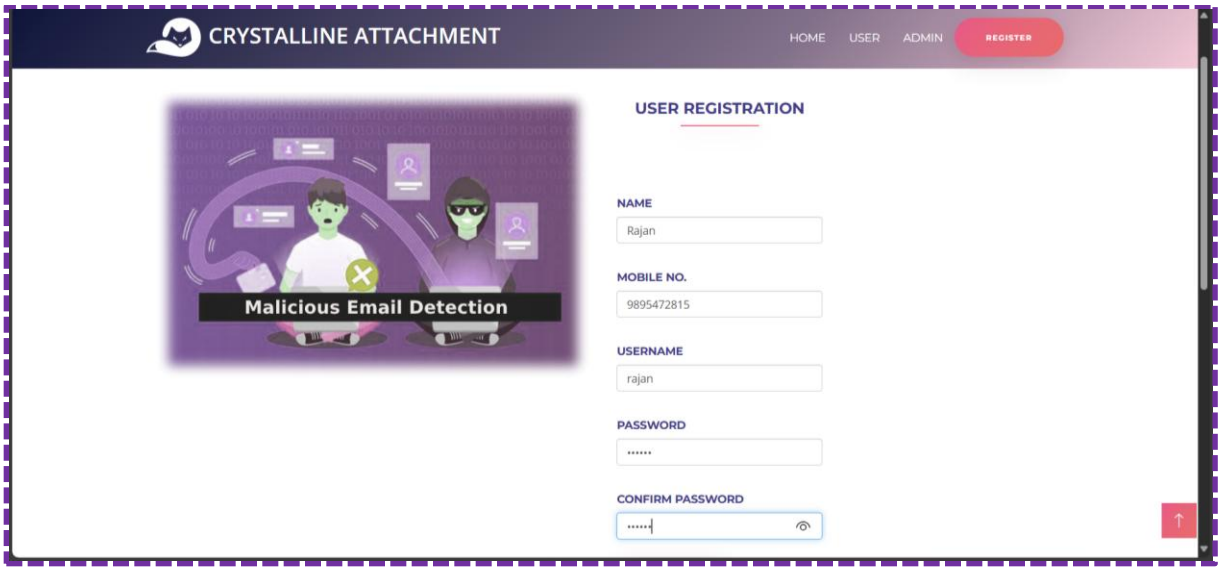

SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply HL 4 info
['six', 'chance', 'win', 'cash', '100', '20', '000', 'pound', 'txt', 'csh11', 'send', '87575', 'cost', '150p', 'day', '6day', '16', 'tsandc', 'apply', 'reply', 'hl', '4', 'info']
URGENT! You have won a 1 week FREE membership in our â£100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18
['urgent', '1', 'week', 'free', 'membership', '100', '000', 'prize', 'jackpot', 'txt', 'word', 'claim', '81010', 'c', 'wwwdbuknet', 'lccltd', 'pobox', '4403ldnw1a7rw18']
I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. You have been wonderful and a blessing at all times.
['searching', 'right', 'word', 'thank', 'breather', 'promise', 'wont', 'take', 'help', 'granted', 'fulfil', 'promise', 'wonderful', 'blessing', 'time']
Remove Punctuation/Stop Words/Tiny Words
Text
Stemmed
Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
go jurong point crazy available bugi great world la e buffet cine got amore wat
Ok lar... Joking wif u oni...
ok lar joking wif oni
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
free entry 2 wkly comp win fa cup final tkt 21st may 2005 text fa 87121 receive entry question std txt rate c apply 08452810075over18
U dun say so early hor... U c already then say...
dun say early hor c already say
Nah I don't think he goes to usf, he lives around here though
nah think goe usf live around though
FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, â£1.50 to rcv
freemsg hey darling 3 week now word back d fun still tb ok xxx std chg send 1 50 rcv
Feature Extraction

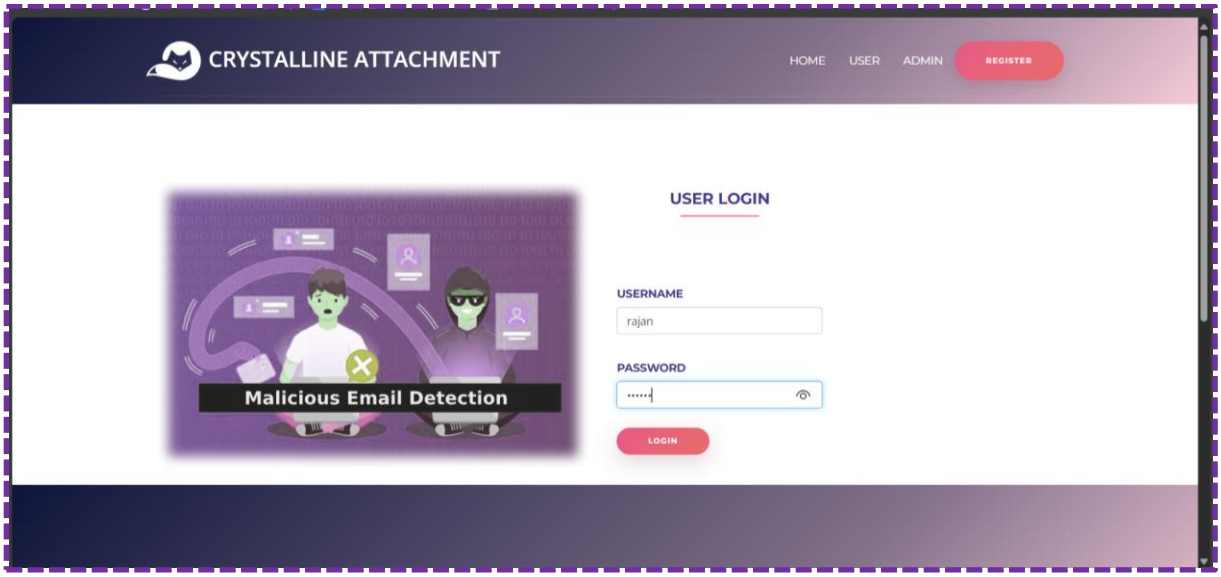

Admin
Dashboard
Logout
Admin
Dashboard
Feature Extraction
TF-IDF
got
free
go
dont
ur
u
im
get
call
know
ok
good
time
later
back
think
like
send
text
come
day
still
one
hi
tgt
reply
stop
txt
mobile
love
want
ill
need
take
4
see
hope
week
home
please
phone
going
well
sorry
thanks
tell
cant
lor
already
great
make
tomorrow
oh
new
way
much
anything
yes
give
number
claim
today
wat
say
dear
night
hey
ask
sure
babe
happy
really
would
meet
prize
pls
didnt
work

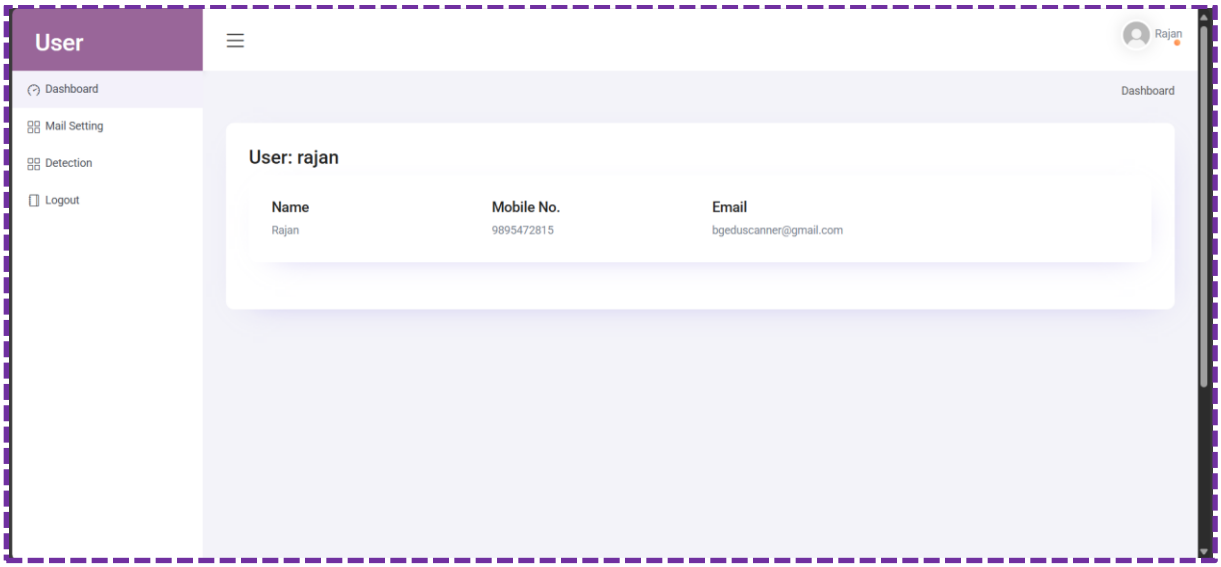

Distribution of number of characters in messages
Frequency
Number of characters in the message
0
200
400
600
800
0
100
200
300
400
500
Top Bigrams in Spam Messages
http www
web site
email address
search engine
forward looking
mailing list
would like
submit website
looking statement
tue jul
adobe premiere
best regard
please click
wish receive
adobe photoshop
jul received
order report
investment advice
per dose
soft viagra
mailwisconsin com
within email
broadcast email
oil gas
business day
N-gram
Frequency
0
50
100
150
200
250
300
Top Bigrams in Non-Spam Messages
hou ect
ect ect
vince kaminski
enron com
cc subject
kaminski hou
ect cc
enron enron
let know
would like
shirley crenshaw
enron communication
corp enron
please let
kaminski enron
com cc
ect subject
hou ect
research group
enron development
http www
risk management
cc vince
stinson gibner
ect pm
N-gram
Frequency
0
1000
2000
3000
4000
5000

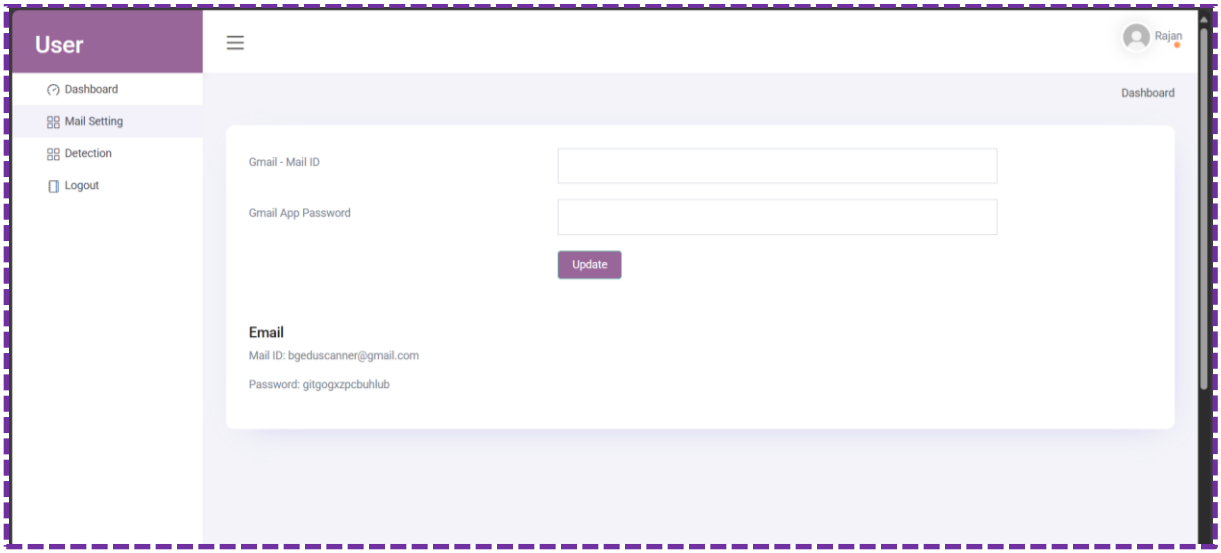

Admin
Dashboard
Logout
Admin
Dashboard
Classification using RoBERTa
Text Class
1 Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat... ham
2 Ok lar... Joking wif u oni... ham
3 Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's spam
4 U dun say so early hor... U c already then say... ham
5 Nah I don't think he goes to usf, he lives around here though ham
6 FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, å£1.50 to rcv spam
7 Even my brother is not like to speak with me. They treat me like aids patent. ham
8 As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune ham
Count
4000
3000
2000

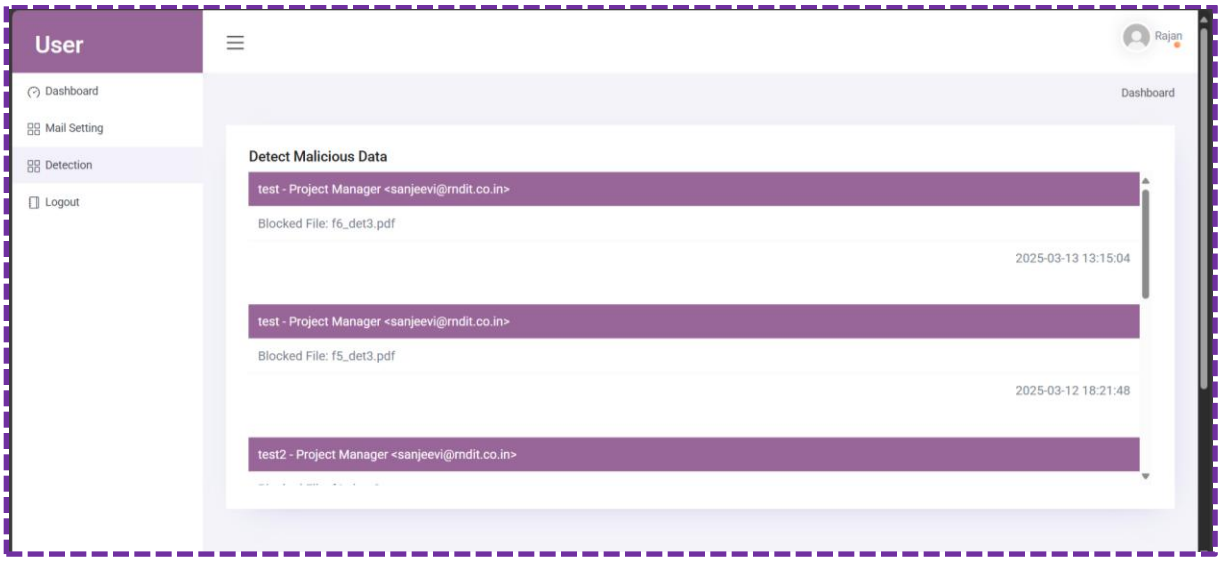

31 Wait that's still not all that clear, were you not sure about me being sarcastic or that that's why x doesn't want to live with us ham
32 Yeah he got in at 2 and was v apologetic. n had fallen out and she was actin like spoilt child and he got caught up in that. Till 2! But we won't go there! Not doing too badly cheers. You? ham
33 K tell me anything about you. ham
34 For fear of fainting with the of all that housework you just did? Quick have a cuppa ham
35 Thanks for your subscription to Ringtone UK your mobile will be charged å£5/month Please confirm by replying YES or NO. If you reply NO you will not be charged spam
36 Yup... Ok i go home look at the timings then i msg Ì_ again... Xuhui going to learn on 2nd may too but her lesson is at 8am ham
Count
4000
3000
2000
1000
Not Spam
Spam
Class
Crystalline Attachment

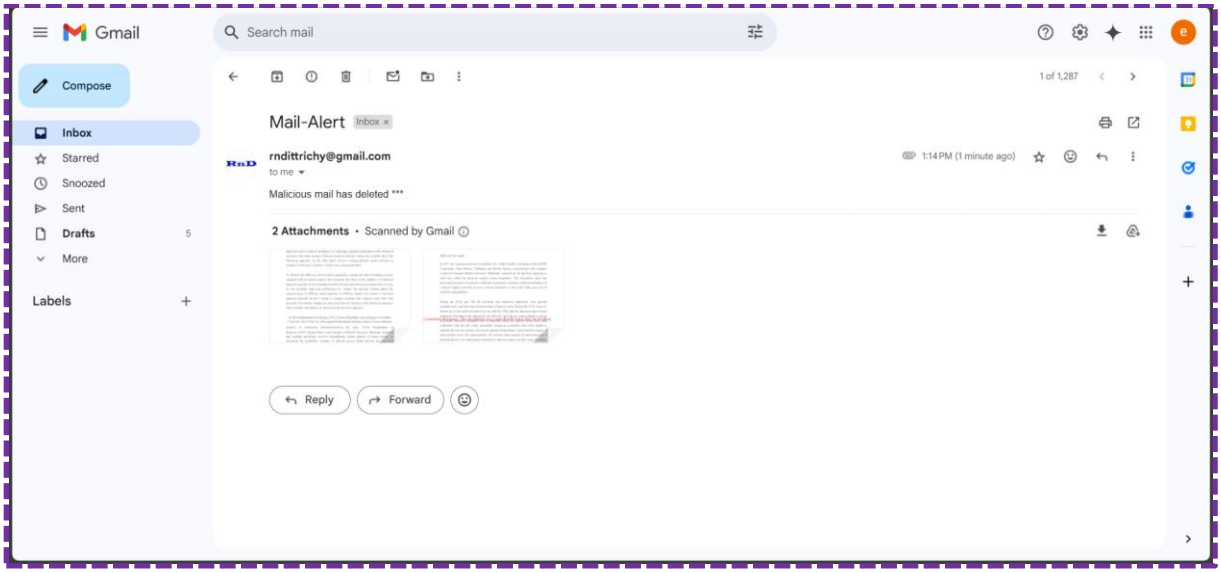

CRYSTALLINE ATTACHMENT
HOME
USER
ADMIN
REGISTER
Malicious Email Detection
USER REGISTRATION
NAME
Rajan
MOBILE NO.
9895472815
USERNAME
rajan
PASSWORD
CONFIRM PASSWORD


CRYSTALLINE ATTACHMENT
HOME
USER
ADMIN
REGISTER
Malicious Email Detection
USER LOGIN
USERNAME
rajan
PASSWORD
LOGIN


User
Rajan
Dashboard
Mail Setting
Detection
Logout
Dashboard
User: rajan
Name
Rajan
Mobile No.
9895472815
Email
bgeduscanner@gmail.com


User
Rajan
Dashboard
Mail Setting
Detection
Logout
Dashboard
Gmail - Mail ID
Gmail App Password
Update
Email
Mail ID: bgeduscanner@gmail.com
Password: gitgogxzpcbuhlub


User
Dashboard
Mail Setting
Detection
Logout
Rajan
Dashboard
Detect Malicious Data
test - Project Manager <sanjeevi@rndit.co.in>
Blocked File: f6_det3.pdf
2025-03-13 13:15:04
test - Project Manager <sanjeevi@rndit.co.in>
Blocked File: f5_det3.pdf
2025-03-12 18:21:48
test2 - Project Manager <sanjeevi@rndit.co.in>


Gmail
Search mail
Compose
Inbox
Starred
Snoozed
Sent
Drafts
More
Labels
Mail-Alert
rndittrichy@gmail.com
to me
1:14 PM (1 minute ago)
Malicious mail has deleted ***
2 Attachments • Scanned by Gmail
Reply
Forward

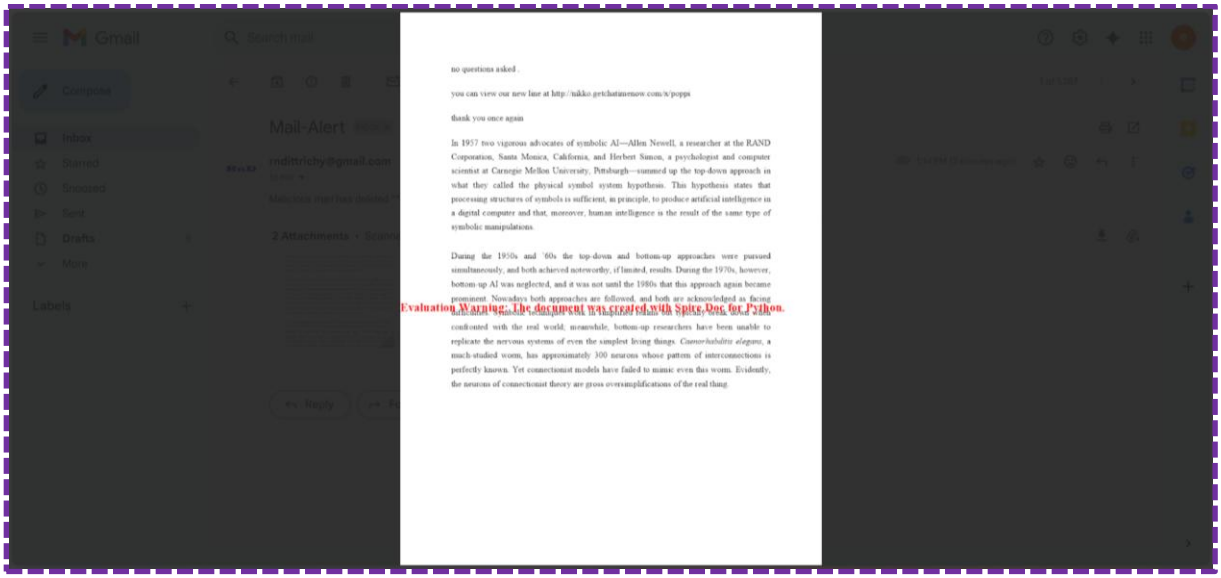
Methods and goals in AI

Symbolic vs. connectionist approaches

AI research follows two distinct, and to some extent competing, methods, the symbolic (or"top-down") approach, and the connectionist (or "bottom-up") approach. The top-down approach seeks to replicate intelligence by analyzing cognition independent of the biological structure of the brain, in terms of the processing of symbols—whence the symbolic label. The bottom-up approach, on the other hand, involves creating artificial neural networks in imitation of the brain's structure—whence the connectionist label.

To illustrate the difference between these approaches, consider the task of building a system, equipped with an optical scanner, that recognizes the letters of the alphabet. A bottom-up approach typically involves training an artificial neural network by presenting letters to it one by one, gradually improving performance by "tuning" the network. (Tuning adjusts the responsiveness of different neural pathways to different stimuli.) In contrast, a top-down approach typically involves writing a computer program that compares each letter with geometric descriptions. Simply put, neural activities are the basis of the bottom-up approach, while symbolic descriptions are the basis of the top-down approach.

In The Fundamentals of Learning (1932), Edward Thorndike, a psychologist at Columbia University, New York City, first suggested that human learning consists of some unknown property of connections between neurons in the brain. In The Organization of Behavior (1949), Donald Hebb, a psychologist at McGill University, Montreal, suggested that learning specifically involves strengthening certain patterns of neural activity by increasing the probability (weight) of induced neuron firing between the associated



no questions asked .

you can view our new line at http://nikko.getchatimenow.com/x/popps

thank you once again

In 1957 two vigorous advocates of symbolic AI—Allen Newell, a researcher at the RAND Corporation, Santa Monica, California, and Herbert Simon, a psychologist and computer scientist at Carnegie Mellon University, Pittsburgh—summed up the top-down approach in what they called the physical symbol system hypothesis. This hypothesis states that processing structures of symbols is sufficient, in principle, to produce artificial intelligence in a digital computer and that, moreover, human intelligence is the result of the same type of symbolic manipulations.

During the 1950s and '60s the top-down and bottom-up approaches were pursued simultaneously, and both achieved noteworthy, if limited, results. During the 1970s, however, bottom-up AI was neglected, and it was not until the 1980s that this approach again became prominent. Nowadays both approaches are followed, and both are acknowledged as facing

confronted with the real world, meanwhile, bottom-up researchers have been unable to replicate the nervous systems of even the simplest living things. *Caenorhabditis elegans*, a much-studied worm, has approximately 300 neurons whose pattern of interconnections is perfectly known. Yet connectionist models have failed to mimic even this worm. Evidently, the neurons of connectionist theory are gross oversimplifications of the real thing.

8.2. SOURCE CODE

Packages

```
from flask import Flask, render_template, Response, redirect, request, session, abort, url_for
from flask_mail import Mail, Message
from flask import send_file
import mysql.connector
import hashlib
from random import randint
from urllib.request import urlopen
import docx
from docx import Document
from pdf2docx import parse
from typing import Tuple
from PIL import Image, ImageDraw, ImageFont
import textwrap
from spire.doc import *
from spire.doc.common import *
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns
from collections import OrderedDict
import re    # for regular expressions
import gensim
from gensim.parsing.preprocessing import remove_stopwords, STOPWORDS
from gensim.parsing.porter import PorterStemmer
import email.policy
from bs4 import BeautifulSoup
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
import sklearn
#from imblearn.over_sampling import SMOTEN
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.metrics import classification_report, plot_confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from wordcloud import WordCloud
from collections import Counter
#Email Configuration
mail_settings = {
"MAIL_SERVER": 'smtp.gmail.com',
"MAIL_PORT": 465,
"MAIL_USE_TLS": False,
"MAIL_USE_SSL": True,
"MAIL_USERNAME": "rndittrichy@gmail.com",
"MAIL_PASSWORD": "lyylfimewwddjwnk"
```

Database Connection

```
mydb = mysql.connector.connect(
host="localhost",
user="root",
password="",
charset="utf8",
database="malicious_email"
```

Login

```
def login():
msg=""
act=request.args.get("act")
if request.method=='POST':
```

```
uname=request.form['uname']
pwd=request.form['pass']
cursor = mydb.cursor()
cursor.execute('SELECT * FROM admin WHERE username = %s AND password = %s',
(uname, pwd))
account = cursor.fetchone()
if account:
session['username'] = uname
return redirect(url_for('admin'))
else:
msg = 'Incorrect username/password!'
```

User Registration

```
def register():
msg=""
act=request.args.get("act")
if request.method=='POST':
name=request.form['name']
mobile=request.form['mobile']
uname=request.form['uname']
pass1=request.form['pass']
mycursor = mydb.cursor()
mycursor.execute("SELECT count(*) FROM register where uname=%s",(uname,))
cnt = mycursor.fetchone()[0]
if cnt==0:
mycursor.execute("SELECT max(id)+1 FROM register")
maxid = mycursor.fetchone()[0]
if maxid is None:
maxid=1
sql = "INSERT INTO register(id,name,mobile,uname,pass) VALUES (%s, %s, %s, %s, %s)"
val = (maxid,name,mobile,uname,pass1)
mycursor.execute(sql, val)
```

```
mydb.commit()
#print(mycursor.rowcount, "Registered Success")
msg="success"
#if mycursor.rowcount==1:
return redirect(url_for('register',act='1'))
else:
msg='fail'
```

Training

```
def load_data():
msg=""
uname=""
if 'username' in session:
uname = session['username']
ff=open("user.txt","r")
uname=ff.read()
ff.close()
act = request.args.get('act')
pd.set_option("display.max_colwidth", 200)
warnings.filterwarnings("ignore") #ignore warnings
data = pd.read_csv(
"static/dataset/spam.csv",
header=0,
encoding="latin-1",
usecols=[0, 1],
names=["label", "text"],
data1=[]
i=0
for ds in data.values:
if i<=200:
data1.append(ds)
i+=1
```

```
#NLP
def lower(text):
return text.lower()
def remove_specChar(text):
return re.sub("#[A-Za-z0-9_]+", ' ', text)
def remove_link(text):
return re.sub('@\S+|https?:\S+|http?:\S|[^A-Za-z0-9]+', ' ', text)
def remove_stopwords(text):
return " ".join([word for word in
str(text).split() if word not in STOPWORDS])
def stemming(text):
return " ".join([stemmer.stem(word) for word in text.split()])
def stem_s(word):
ww=word.split(" ")
wd=[]
for wr in ww:
w1=len(wr)-1
w2=len(wr)
wrr=wr[w1:w2]
if wrr == 's':
wd.append(wr[:-1])
else:
wd.append(wr)
res=" ".join(wd)
return res
def lemmatizer_words(text):
return " ".join([lematizer.lemmatize(word) for word in text.split()])
def cleanTxt(text):
text = lower(text)
text = remove_specChar(text)
```

```
text = remove_link(text)
text = remove_stopwords(text)
text = stemming(text)
text = stem_s(text)
return text
def generate_token(query):
q4=[]
query1=""
q1=query.split(" ")
for q11 in q1:
q2=q11.split(".")
q3="".join(q2)
q4.append(q3)
query1=" ".join(q4)
text=cleanTxt(query1)
doc=text.split(" ")
text_tokens=text.split(" ")
tokens_without_sw = [word for word in text_tokens if not word in stop_words]
return tokens_without_sw
#Preprocessing
def preprocess():
data1=[]
data2=[]
data3=[]
df = pd.read_csv(
"static/dataset/spam.csv",
header=0,
encoding="latin-1",
usecols=[0, 1],
names=["label", "text"],
i=0
```

```
for ds in df.values:
dt1=[]
if i<200:
if ds[1]=="":
s=1
else:
dt1.append(ds[1])
dd=generate_token(ds[1])
dt1.append(dd)
data1.append(dt1)
i+=1
i=0
for ds2 in df.values:
dt2=[]
if i<200:
if ds2[1]=="":
s=1
else:
dt2.append(ds2[1])
dd2=cleanTxt(ds2[1])
dt2.append(dd2)
data2.append(dt2)
i+=1
#Feature Extraction
def feature():
data1=[]
'"emails = pd.read_csv(
"static/dataset/spam.csv",
header=0,
encoding="latin-1",
usecols=[0, 1],
```

```
names=["label", "text"],
emails= pd.read_csv("static/dataset/spam.csv", encoding="ISO-8859-1")
emails.shape
emails.head()
emails = emails.rename(columns={"v1": "Classify", "v2": "Email"})
print(emails.isnull().sum())
print("Total Duplicates:", emails.duplicated().sum())
emails = emails.drop_duplicates()
print("Total Duplicates:", emails.duplicated().sum())
total_messages = len(emails)
print("Total number of messages:", total_messages)
'''spam_count = len(emails[emails['Classify'] == 'spam'])
ham_count = len(emails[emails['Classify'] == 'ham'])
total_messages = len(emails)
spam_percentage = (spam_count / total_messages) * 100
ham_percentage = (ham_count / total_messages) * 100
# Create a pie chart
labels = ['Spam', 'Regular (ham)']
sizes = [spam_percentage, ham_percentage]
colors = ['#ff9999', '#66b3ff']
explode = (0.1, 0)  # Explode the first slice (spam)'''
# Remove punctuation marks
emails['Email'] = emails['Email'].apply(lambda x: x.translate(str.maketrans('', '',
string.punctuation)))
# Convert email text to lowercase
emails['Email'] = emails['Email'].str.lower()
# Remove stopwords
#stop_words = set(stopwords.words('english'))
emails['Email'] = emails['Email'].apply(lambda x: ' '.join([word for word in x.split() if word not
in stop_words]))
# Split each email into individual words
```

```
all_words = ' '.join(emails['Email']).split()
# Count the frequency of each word
word_counts = Counter(all_words)
# Get the most common words
most_common_words = word_counts.most_common(100)
print (most_common_words[:10])
# Create a word cloud object
wordcloud = WordCloud(width=800, height=400, background_color='white')
# Generate the word cloud from the most_common_words list
wordcloud.generate_from_frequencies(dict(most_common_words))
# Plot the word cloud
plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
#plt.savefig('static/graph/ff.png')
plt.close()
#plt.show()
# Filter regular messages (ham)
regular_messages = emails[emails['Classify'] == 'ham']
# Count the number of regular messages containing the word "hello"
count_hello = regular_messages['Email'].str.contains('hello', case=False).sum()
# Print the result
print("Number of regular messages containing the word 'hello':", count_hello)
import re
phone_number_count = 0
for message in emails.loc[emails['Classify'] == 'spam', 'Email']:
if re.search(r'\b\d{10}\b', message):  # assuming phone numbers are 10 digits long
phone_number_count += 1
print("Number of spam messages containing a phone number:", phone_number_count)
word_counts = emails['Email'].apply(lambda x: len(str(x).split()))
average_word_count = word_counts.mean()
```

```
emails['number_of_characters_in_the_message'] = emails['Email'].apply(len)
emails['number_of_characters_in_the_message'].hist(bins=50)
plt.xlabel('Number of characters in the message')
plt.ylabel('Frequency')
plt.title('Distribution of number of characters in messages')
plt.savefig('static/graph/ff2.png')
# Filter spam and non-spam messages
'''spam_texts = emails[emails['Classify'] == 'spam']['Email']
non_spam_texts = emails[emails['Classify'] == 'ham']['Email']
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
plot_ngrams(axes[0], spam_texts, title='Top Bigrams in Spam Messages')
plot_ngrams(axes[1], non_spam_texts, title='Top Bigrams in Non-Spam Messages')
axes[0].grid(axis='x')
axes[1].grid(axis='x')
plt.tight_layout()
plt.show()
vectorizer = CountVectorizer()
# Bag of words
bow_text = vectorizer.fit_transform(emails["Email"])
transformed_bow = vectorizer.transform(emails["Email"])
# TF-IDF
tfidf_transformer = TfidfTransformer().fit(transformed_bow)
text_tfidf = tfidf_transformer.transform(transformed_bow)
#DistilBERT--Malicious Content Detection
def DistilBERT():
batch = collate_tokens(
[DistilBERT.encode(pair[0], pair[1]) for pair in batch_of_pairs], pad_idx=1
logprobs = DistilBERT.predict('mnli', batch)
print(logprobs.argmax(dim=1))
label_map = {0: 'contradiction', 1: 'neutral', 2: 'entailment'}
ncorrect, nsamples = 0, 0
```

```
DistilBERT.cuda()
DistilBERT.eval()
with open('static/dataset/spam11.csv') as fin:
fin.readline()
for index, line in enumerate(fin):
tokens = line.strip().split('\t')
sent1, sent2, target = tokens[8], tokens[9], tokens[-1]
tokens = DistilBERT.encode(sent1, sent2)
prediction = DistilBERT.predict('mnli', tokens).argmax().item()
prediction_label = label_map[prediction]
ncorrect += int(prediction_label == target)
nsamples += 1
print('| Accuracy: ', float(ncorrect)/float(nsamples))
def get_data():
text = str(self.text[index])
text = " ".join(text.split())
inputs = self.tokenizer.encode_plus(
text,
None,
add_special_tokens=True,
max_length=self.max_len,
pad_to_max_length=True,
return_token_type_ids=True
ids = inputs['input_ids']
token_type_ids = inputs["token_type_ids"]
return {
'ids': torch.tensor(ids, dtype=torch.long),
'token_type_ids': torch.tensor(token_type_ids, dtype=torch.long),
'targets': torch.tensor(self.targets[index], dtype=torch.float)
train_size = 0.8
train_data=new_df.sample(frac=train_size,random_state=200)
```

```
test_data=new_df.drop(train_data.index).reset_index(drop=True)
train_data = train_data.reset_index(drop=True)
print("FULL Dataset: {}".format(new_df.shape))
print("TRAIN Dataset: {}".format(train_data.shape))
print("TEST Dataset: {}".format(test_data.shape))
training_set = SentimentData(train_data, tokenizer, MAX_LEN)
testing_set = SentimentData(test_data, tokenizer, MAX_LEN)
train_params = {'batch_size': TRAIN_BATCH_SIZE,
'shuffle': True,
'num_message': 0
test_params = {'batch_size': VALID_BATCH_SIZE,
'shuffle': True,
'num_message': 0
training_loader = DataLoader(training_set, **train_params)
testing_loader = DataLoader(testing_set, **test_params)
def train(epoch):
tr_loss = 0
n_correct = 0
nb_tr_steps = 0
nb_tr_examples = 0
model.train()
for _,data in tqdm(enumerate(training_loader, 0)):
ids = data['ids'].to(device, dtype = torch.long)
mask = data['mask'].to(device, dtype = torch.long)
token_type_ids = data['token_type_ids'].to(device, dtype = torch.long)
targets = data['targets'].to(device, dtype = torch.long)
outputs = model(ids, mask, token_type_ids)
loss = loss_function(outputs, targets)
tr_loss += loss.item()
big_val, big_idx = torch.max(outputs.data, dim=1)
n_correct += calcuate_accuracy(big_idx, targets)
```

```
nb_tr_steps += 1
nb_tr_examples+=targets.size(0)
if _%5000==0:
loss_step = tr_loss/nb_tr_steps
accu_step = (n_correct*100)/nb_tr_examples
print(f"Training Loss per 5000 steps: {loss_step}")
print(f"Training Accuracy per 5000 steps: {accu_step}")
optimizer.zero_grad()
loss.backward()
# # When using GPU
optimizer.step()
print(f'The Total Accuracy for Epoch {epoch}: {(n_correct*100)/nb_tr_examples}')
epoch_loss = tr_loss/nb_tr_steps
epoch_accu = (n_correct*100)/nb_tr_examples
print(f"Training Loss Epoch: {epoch_loss}")
print(f"Training Accuracy Epoch: {epoch_accu}")
return
def valid(model, testing_loader):
model.eval()
n_correct = 0; n_wrong = 0; total = 0; tr_loss=0; nb_tr_steps=0; nb_tr_examples=0
with torch.no_grad():
for _, data in tqdm(enumerate(testing_loader, 0)):
ids = data['ids'].to(device, dtype = torch.long)
mask = data['mask'].to(device, dtype = torch.long)
token_type_ids = data['token_type_ids'].to(device, dtype=torch.long)
targets = data['targets'].to(device, dtype = torch.long)
outputs = model(ids, mask, token_type_ids).squeeze()
loss = loss_function(outputs, targets)
tr_loss += loss.item()
big_val, big_idx = torch.max(outputs.data, dim=1)
n_correct += calcuate_accuracy(big_idx, targets)
```

```
nb_tr_steps += 1
nb_tr_examples+=targets.size(0)
if _%5000==0:
loss_step = tr_loss/nb_tr_steps
accu_step = (n_correct*100)/nb_tr_examples
print(f"Validation Loss per 100 steps: {loss_step}")
print(f"Validation Accuracy per 100 steps: {accu_step}")
epoch_loss = tr_loss/nb_tr_steps
epoch_accu = (n_correct*100)/nb_tr_examples
print(f"Validation Loss Epoch: {epoch_loss}")
print(f"Validation Accuracy Epoch: {epoch_accu}")
return epoch_accu
```

Test Mail

```
def convert_pdf2docx(input_file: str, output_file: str, pages: Tuple = None):
"""Converts pdf to docx"""
if pages:
pages = [int(i) for i in list(pages) if i.isnumeric()]
result = parse(pdf_file=input_file,
docx_with_path=output_file, pages=pages)
summary = {
"File": input_file, "Pages": str(pages), "Output File": output_file
print("\n".join("{}:{}".format(i, j) for i, j in summary.items()))
return result
def text_to_image(text_file, fid):
image_file="m"+str(fid)+"_1.png"
font_path="static/arial.ttf"
font_size=20
wrap_width=80
with open("static/attachments/"+text_file, 'r') as f:
text = f.read()
# Wrap the text
```

```
wrapped_text = textwrap.wrap(text, width=wrap_width)
# Create a new image with appropriate dimensions
line_height = font_size + 5  # Add some spacing between lines
img_width = max(len(line) * font_size for line in wrapped_text) * 2
img_height = len(wrapped_text) * line_height
img = Image.new("RGB", (img_width, img_height), "white")
draw = ImageDraw.Draw(img)
# Load the font
font = ImageFont.truetype(font_path, font_size)
# Draw the text on the image
y_text = 0
for line in wrapped_text:
draw.text((50, y_text), line, font=font, fill="black")
y_text += line_height
# Save the image
img.save("static/attachments/"+image_file)
def word_to_img(wfile,fid):
# Create a Document object
document = Document()
# Load a Word DOCX file
document.LoadFromFile("static/attachments/"+wfile)
# Or load a Word DOC file
document.LoadFromFile("Sample.doc")
# Convert the document to a list of image streams
image_streams = document.SaveImageToStreams(ImageType.Bitmap)
# Incremental counter
i = 1
# Save each image stream to a PNG file
for image in image_streams:
image_name = "m"+str(fid)+"_"+str(i) + ".png"
with open("static/attachments/"+image_name,'wb') as image_file:
```

```
image_file.write(image.ToArray())
i += 1
# Close the document
document.Close()
return i
def emailsink(usermail,pwd,uname):
mycursor = mydb.cursor()
import email
import imaplib
mail = imaplib.IMAP4_SSL('imap.gmail.com')
(retcode, capabilities) = mail.login(usermail,pwd)
mail.list()
mail.select('inbox')
subj1="Spam"
(retcode, messages) = mail.search(None, '(UnSeen)')
if retcode == 'OK':
for num in messages[0].split() :
print ('Processing ')
n=n+1
mycursor.execute("SELECT max(id)+1 FROM read_data")
maxid = mycursor.fetchone()[0]
if maxid is None:
maxid=1
if n<4:
typ, data = mail.fetch(num,'(RFC822)')
for response_part in data:
if isinstance(response_part, tuple):
original = email.message_from_bytes(response_part[1])
raw_email = data[0][1]
raw_email_string = raw_email.decode('utf-8')
email_message = email.message_from_string(raw_email_string)
```

```
for part in email_message.walk():
if part.get_content_maintype() == 'multipart':
#print(part.as_string())
print("a")
fu="1"
continue
if part.get('Content-Disposition') is None:
#print(part.as_string())
print("b")
continue
if fu=="1":
fileName = part.get_filename()
print('file names processed ...')
print(fileName)
j+=1
f1=fileName.split(".")
if f1[1]=="txt" or f1[1]=="docx" or f1[1]=="pdf":
fname="f"+str(maxid)+"_"+fileName
ff+=fname+"|"
fp = open("static/attachments/"+fname, 'wb')
fp.write(part.get_payload(decode=True))
fp.close()
if (part.get_content_type() == "text/plain"): # ignore attachments/html
body = part.get_payload(decode=True)
'''save_string = str(r"data.txt" )
myfile = open(save_string, 'a')
myfile.write(original['From']+'\n')
myfile.write(original['Subject']+'\n')
myfile.write(body.decode('utf-8'))'''
subj=original['Subject']
sender=original['From']
```

```
mess=body.decode('utf-8')
if f1[1]=="txt":
fp1=open("static/testdata.txt","r")
val=fp1.read()
fp1.close()
fp12=open("static/attachments/"+fname,"r")
txt=fp12.read()
fp12.close()
sval=val.split("|")
x=0
for sval2 in sval:
if sval2 in txt:
x+=1
if x>0:
text_to_image(fname,maxid)
n1=1
#Delete mail
mail.store(num,'+FLAGS',r'(\Deleted)')
mess1="Malicious mail has deleted ***"
sendmail(usermail,mess1,maxid,n1)
sender=original['From']
subj=original['Subject']
sql = "INSERT INTO read_data(id,subject,sender,uname,message,spam_st,filename,img_count)
VALUES (%s, %s, %s, %s, %s, %s,%s,%s)"
val = (maxid,subj,sender,uname,'','0',fname,n1)
mycursor.execute(sql, val)
mydb.commit()
print("mail sent")
elif f1[1]=="docx":
doc = docx.Document("static/attachments/"+fname)
all_paras = doc.paragraphs
```

```
fp1=open("static/testdata.txt","r")
val=fp1.read()
fp1.close()
sval=val.split("|")
x=0
for sval2 in sval:
for para in all_paras:
#print(para.text)
txt=para.text
if sval2 in txt:
x+=1
if x>0:
nn=word_to_img(fname,maxid)
n1=nn-1
#Delete mail
mail.store(num,'+FLAGS',r'(\Deleted)')
mess1="Malicious mail has deleted ***"
sendmail(usermail,mess1,maxid,n1)
sender=original['From']
subj=original['Subject']
sql = "INSERT INTO read_data(id,subject,sender,uname,message,spam_st,filename,img_count)
VALUES (%s, %s, %s, %s, %s, %s,%s,%s)"
val = (maxid,subj,sender,uname,'','0',fname,n1)
mycursor.execute(sql, val)
mydb.commit()
print("mail sent")
elif f1[1]=="pdf":
p1=fname.split(".")
fname2=p1[0]+".docx"
convert_pdf2docx("static/attachments/"+fname,"static/attachments/"+fname2)
```

```
doc = docx.Document("static/attachments/"+fname2)
all_paras = doc.paragraphs
fp1=open("static/testdata.txt","r")
val=fp1.read()
fp1.close()
sval=val.split("|")
x=0
for sval2 in sval:
for para in all_paras:
#print(para.text)
txt=para.text
if sval2 in txt:
x+=1
if x>0:
nn=word_to_img(fname2,maxid)
n1=nn-1
#Delete mail
mail.store(num,'+FLAGS',r'(\Deleted)')
mess1="Malicious mail has deleted ***"
sendmail(usermail,mess1,maxid,n1)
sender=original['From']
subj=original['Subject']
sql = "INSERT INTO read_data(id,subject,sender,uname,message,spam_st,filename,img_count)
VALUES (%s, %s, %s, %s, %s, %s,%s,%s)"
val = (maxid,subj,sender,uname,'','0',fname,n1)
mycursor.execute(sql, val)
mydb.commit()
print("mail sent")
```

CHAPTER 9

BOTTOMLINE

9.1. CONCLUSION

In conclusion, the project is designed with a security-first approach, leveraging Python with Flask for Back-End processing and React JS for a dynamic Front-End interface. The integration of MySQL ensures structured and efficient data management, handling user accounts, email metadata, and processed attachments. The system's core functionalities, including CANet model-based malicious attachment detection, NLP-powered preprocessing, TF-IDF feature extraction, and DistilBERT-based classification, enhance the security of email communication. Key modules such as Admin Management, User Authentication, Dataset Upload, Model Training, Email Processing, Attachment Analysis, Threat Detection, Alert System, and Safe Attachment Conversion work cohesively to provide a secure and efficient email experience. The attachment conversion mechanism, which transforms suspicious attachments into static PNG images, adds an extra layer of security, ensuring users can access content without the risk of executing malicious code. The real-time security alerts and warnings further empower users to make informed decisions while handling email attachments. With its intelligent threat detection, seamless user experience, and robust security measures, the project presents a comprehensive solution for safeguarding email attachments from cyber threats. It stands as a proactive cybersecurity tool, revolutionizing traditional email security mechanisms by integrating machine learning, NLP, and real-time alerts into a user-friendly and scalable web application.

9.2. FUTURE ENHANCEMENT

To further improve the project, several future enhancements can be considered:

- **Multi-language Support:** Expand the system to support multiple languages for broader usability and accessibility in international markets.
- **Blockchain-based Security:** Investigate the use of blockchain technology to enhance the security and immutability of attachment processing and data storage.
- **Integration with IoT Devices:** Explore integration with IoT devices for secure handling of email attachments in smart devices and connected environments.

CHAPTER 10

REFERENCE

10.1. JOURNAL REFERENCE

1. L. Gallo, D. Gentile, S. Ruggiero, A. Botta and G. Ventre, "The human factor in phishing: Collecting and analyzing user behavior when reading emails", Comput. Secur., vol. 139, Apr. 2024.
2. M. A. Bouke, A. Abdullah, M. T. Abdullah, S. A. Zaid, H. E. Atigh and S. H. Alshatebi, "A lightweight machine learning-based email spam detection model using word frequency pattern", J. Inf. Technol. Comput., vol. 4, no. 1, pp. 15-28, Jun. 2023.
3. M. Salb, L. Jovanovic, M. Živković, E. Tuba, A. Elsadai and N. Bačanin, "Training logistic regression model by enhanced moth flame optimizer for spam email classification", Proc. 5th Comput. Netw. Inventive Commun. Technol. (ICCNCT), pp. 753-768, Oct. 2022.
4. N. H. Marza, M. E. Manaa and H. A. Lafta, "Classification of spam emails using deep learning", Proc. 1st Babylon Int. Conf. Inf. Technol. Sci. (BICITS), pp. 63-68, Apr. 2021.
5. O. E. Taylor and P. S. Ezekiel, "A model to detect spam email using support vector classifier and random forest classifier", Int. J. Comput. Sci. Math. Theory, vol. 6, no. 1, pp. 1-11, 2020.
6. Andi Fariana and Syauqi Jinan, "The urgency of intellectual property rights in the digital era from the perspective of Sharia economic law in Indonesia", Int. J. Res. Bus. Soc. Sci. (2147-4478), vol. 12, no. 8, pp. 552-556, 2023.
7. P. Kumar, "Intellectual Property Rights (IPR): Nurturing Creativity Fostering Innovation", vol. 02, no. 02, pp. 32-38, 2024.
8. F. Indra and F. Santiago, "Intellectual Property Rights in Legal Perspective in Indonesia", Proc. First Multidiscip. Int. Conf., 2022.
9. X. Sun, X. Zhou, Q. Wang, P. Tang, E. L. C. Law and S. Cobb, "Understanding attitudes towards intellectual property from the perspective of design professionals", Electron. Commer. Res., vol. 21, no. 2, pp. 521-543, 2021.

10.2. BOOK REFERENCE

- Python – Python Crash Course by Eric Matthes
- React JS – Learning React by Alex Banks and Eve Porcello
- MySQL – MySQL 8.0 Reference Manual by Oracle Corporation
- WAMP Server – WAMP Server User Guide by WAMP Community
- TensorFlow – Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron
- Bootstrap – Bootstrap 5 By Example by Silvio Moreto
- Flask – Flask Web Development by Miguel Grinberg

10.3. WEB REFERENCE

- Python – https://docs.python.org/3/
- React JS – https://react.dev/
- MySQL – https://dev.mysql.com/doc/
- WAMP Server – https://www.wampserver.com/en/
- TensorFlow – https://www.tensorflow.org/
- Bootstrap – https://getbootstrap.com/
- Flask – https://flask.palletsprojects.com/